SIMATS ENGINEERING

Department of Artificial Intelligence And Machine Learning (AIML)

ITA0507 – Computer Vision

COMPUTER VISION – ASSIGNMENT 2 (CO2)

OLD PHOTOGRAPH RESTORATION SYSTEM

CO2: Apply image enhancement and segmentation techniques to restore damaged photographs and identify meaningful image regions.

Assignment Details	Information	Assignment Details	Information
Student Name	P.Abdul	Register Number	192525426
Department / Section	AIML	Academic Year / Semester	2025–2026
Faculty Name	Murali P	Nature of Assignment	Individual
Duration	1 Week	Total Marks	50
Date of Submission	5.11.2026	Date of Evaluation	07.11.2026

SCENARIO

A college archive has a collection of old family and institutional photographs that have become faded and damaged over time. The photographs show loss of contrast, yellow/sepia colour shifts, scratches, dust, stains, ink marks, creases and uneven fading. Some images are slightly blurred, while others contain small torn or missing regions. The archive wants an Old Photograph Restoration System that (1) improves brightness, contrast and visual clarity while preserving original facial features and fine details, and (2) identifies damaged regions such as scratches, stains, dust spots, creases and torn areas so that they can be repaired or reviewed. The restored result must retain the historical appearance of the photograph and must not invent or alter important details.

You are the vision engineer. For every task, three candidate techniques are provided. Implement all three techniques on the same set of old photographs, compare their performance, select the most suitable technique, and justify the selection with experimental evidence. Pay special attention to preserving faces, clothing, background details and original photographic tone. State and justify any additional assumptions.

OPERATING DATA

Parameter	Operating Data
Image resolution	A4, 2480 × 3508 pixels (300 DPI)
Image format	8-bit grayscale
Text stroke width	2–4 pixels
Contrast	Faded ink is 30–50 grey levels darker than paper
Lighting	Uneven illumination with one-sided shadows
Noise	Gaussian noise ($\sigma \approx 6$) and 1% salt-and-pepper noise
Scanner noise	Horizontal stripes every 6 pixel rows
Processing time	Maximum 3 seconds per page
Target accuracy	Text segmentation Dice score ≥ 0.90
Hardware	4 CPU cores, 16 GB RAM, no GPU
Software	Python, OpenCV and NumPy

TASK 1 – GREY LEVEL TRANSFORMATION ANALYSIS

1. INTRODUCTION

Grey Level Transformation is an important image processing technique used to improve the visual quality of digital photographs. It modifies the intensity values of pixels to improve brightness, contrast and visibility of important image details.

Old photographs often suffer from fading, yellowing, uneven illumination, low contrast, scratches, dust and other forms of physical damage. Due to these problems, important details such as facial features, clothing patterns and background information may become difficult to identify.

Grey level transformation techniques help improve the appearance of these photographs by changing their pixel intensity values. These techniques are especially useful when the original image contains faded or poorly visible details.

In this task, three candidate techniques are studied:

1. Negative and Log Transformation.
2. Power-Law (Gamma) Transformation.
3. Piecewise-Linear Contrast Stretching.

All three techniques are implemented using Python, OpenCV and NumPy. Their performance is compared based on subject-to-background contrast, recovery of faint details, clipping effects and processing time.

Bit-plane slicing is also performed to identify which binary planes contain useful image information and which mainly contain noise or damage.

2. OBJECTIVES

1. To understand the concept of grey level transformation.
2. To implement negative transformation using Python and OpenCV.
3. To implement logarithmic transformation.
4. To implement power-law or gamma transformation.
5. To implement piecewise-linear contrast stretching.
6. To compare the contrast improvement of different techniques.
7. To study the recovery of faint details in old photographs.
8. To analyse clipping effects caused by intensity transformations.
9. To plot transformation curves and input/output histograms.
10. To perform bit-plane slicing and analyse image information.

11. To understand the effect of applying the same gamma value to bright and dark photographs.
12. To compare traditional transformations with pretrained image-enhancement models.

3. GREY LEVEL TRANSFORMATION FUNDAMENTALS

Grey level transformation is a point-processing technique in which each input pixel intensity is converted into a corresponding output intensity.

The general mathematical representation is:

$s = T(r)$

Where:

- r = Input pixel intensity.
- s = Output pixel intensity.
- T = Transformation function.

For an 8-bit grayscale image, pixel values range from 0 to 255.

- 0 represents black.
- 255 represents white.
- Values between 0 and 255 represent different shades of grey.

The purpose of grey level transformation is to modify the intensity distribution to make important image details more visible.

3.1 Importance of Grey Level Transformation

1. Improves image brightness.
2. Enhances low-contrast photographs.
3. Helps reveal faint image details.
4. Improves the visibility of dark regions.
5. Supports old photograph restoration.
6. Helps prepare images for further processing.
7. Makes important visual information easier to identify.

4. TECHNIQUE A – NEGATIVE AND LOG TRANSFORMATION

4.1 Negative Transformation

Definition

Negative transformation is a grey level transformation technique that reverses the intensity values of an image.

Dark pixels become bright, and bright pixels become dark. It produces an image that looks like a photographic negative.

This technique is useful for examining details that may be difficult to observe in the original image.

Mathematical Formula

For an 8-bit grayscale image:

$s = 255 - r$

Where:

- r = Original pixel intensity.
- s = Output pixel intensity.
- 255 = Maximum grayscale intensity.

Example

Input Intensity	Output Intensity
0	255
50	205
100	155
150	105
200	55
255	0

Working Procedure

1. Read the original grayscale photograph.
2. Identify the intensity value of every pixel.
3. Subtract each intensity value from 255.
4. Assign the calculated value to the corresponding output pixel.
5. Display the negative image.
6. Compare the original and transformed photographs.

Advantages

1. Simple to implement.
2. Reverses image intensity values.
3. Can reveal certain details in dark regions.
4. Useful for analysing photographic negatives.
5. Requires low computational resources.

Limitations

1. Reverses the natural appearance of the photograph.
2. Does not directly repair scratches or stains.
3. May make normal facial appearance difficult to interpret.
4. Not generally suitable as the final restoration output.

4.2 Logarithmic Transformation

Definition

Logarithmic transformation is used to expand darker intensity values while compressing higher intensity values.

It is useful when important information is hidden in darker regions of an image.

In old photographs, some faded or poorly illuminated details may be difficult to observe. Log transformation can improve the visibility of these regions.

Mathematical Formula

$s = c \times \log(1 + r)$

Where:

- r = Input pixel intensity.
- s = Output pixel intensity.
- c = Scaling constant.
- log = Natural logarithm.

For an 8-bit image, a commonly used scaling constant is:

$c = 255 / \log(256)$

Therefore:

$s = [255 / \log(256)] \times \log(1 + r)$

The output values are limited to the range 0 to 255.

Working Procedure

1. Read the original grayscale image.
2. Convert the image into a floating-point representation.
3. Add 1 to each pixel value to avoid taking the logarithm of zero.
4. Apply the logarithmic function.
5. Normalize the result to the 0–255 range.
6. Convert the result into an 8-bit image.
7. Display and compare the enhanced image.

Advantages

1. Expands darker intensity values.
2. Helps reveal details in dark regions.
3. Compresses high-intensity values.
4. Can improve the visibility of low-intensity information.
5. Useful for analysing images with a wide intensity range.

Limitations

1. May reduce the contrast of bright regions.
2. Can make the image appear unnatural.

3. May increase the visibility of noise in dark areas.
4. Does not remove physical damage.

5. TECHNIQUE B – POWER-LAW (GAMMA) TRANSFORMATION

5.1 Definition

Power-Law Transformation, also known as Gamma Transformation, is a grey level transformation technique that modifies image brightness using a power function.

It is commonly used to correct brightness and improve the visibility of image details.

The gamma value determines how the input intensity values are changed.

It is useful for old photographs that are too dark or too bright.

5.2 Mathematical Formula

The power-law transformation is:

$s = c \times r^{\gamma}$

Where:

- r = Normalized input intensity.
- s = Normalized output intensity.
- c = Scaling constant.
- γ = Gamma value.

For an 8-bit grayscale image, input values are normalized as:

r = Input Pixel / 255

The output is calculated as:

Output Pixel = 255 × (Input Pixel / 255)^γ

For normal gamma correction, c is taken as 1 in the normalized representation.

5.3 Effect of Gamma Values

The gamma value controls the brightness of the output image.

Gamma Less Than 1

When γ is less than 1, darker pixel values are expanded towards brighter values.

This makes a dark photograph brighter.

Example:

γ = 0.5

Gamma Equal to 1

When γ is equal to 1, the output image remains approximately the same as the input image.

Example:

γ = 1.0

Gamma Greater Than 1

When γ is greater than 1, pixel intensity values are reduced towards darker values.

This makes a bright photograph darker.

Example:

$\gamma = 2.0$

5.4 Example of Gamma Transformation

Gamma Value	Expected Output
0.4	Strong brightening
0.6	Moderate brightening
1.0	No significant change
1.5	Moderate darkening
2.0	Stronger darkening

The actual visual result depends on the original photograph's intensity distribution.

5.5 Working Procedure

1. Load the original grayscale photograph.
2. Normalize pixel values between 0 and 1.
3. Select a gamma value.
4. Apply the power-law equation.
5. Scale the output back to the range 0–255.
6. Convert the output to an 8-bit image.
7. Compare the original and transformed photographs.

5.6 Advantages

1. Provides flexible brightness correction.
2. Can improve dark photographs.
3. Can reduce excessive brightness.
4. Easy to implement.
5. Supports different gamma values for different photographs.
6. Useful for correcting faded image appearance.

5.7 Limitations

1. One gamma value may not work for every photograph.
2. Excessive brightening may amplify noise.
3. Excessive darkening may hide faint details.
4. May change the natural appearance of facial features.
5. Does not repair scratches, stains or missing regions.

6. TECHNIQUE C – PIECEWISE-LINEAR CONTRAST STRETCHING

6.1 Definition

Piecewise-Linear Contrast Stretching is a grey level transformation technique that improves image contrast by mapping selected input intensity ranges to a wider output intensity range.
It is useful for old photographs containing faded details and low contrast between the subject and background.
Unlike a single gamma transformation, piecewise-linear transformation allows different intensity ranges to be adjusted separately.

6.2 Mathematical Formula

A basic piecewise-linear contrast stretching function can be represented using three control points:
(r1, s1), (r2, s2), and the endpoints (0, 0), (255, 255).
The transformation is:
For $0 \leq r \leq r1$:
$s = (s1 / r1) \times r$
For $r1 < r \leq r2$:
$s = s1 + [(s2 - s1) / (r2 - r1)] \times (r - r1)$
For $r2 < r \leq 255$:
$s = s2 + [(255 - s2) / (255 - r2)] \times (r - r2)$
Where:

- r = Input intensity.
- s = Output intensity.
- r1 and r2 = Selected input control points.
- s1 and s2 = Corresponding output control points.

The selected control points determine which intensity ranges receive greater contrast enhancement.

6.3 Working Procedure

1. Load the original grayscale photograph.
2. Analyse the intensity distribution.
3. Select suitable input control points.
4. Select corresponding output control points.
5. Apply the piecewise transformation.
6. Convert the output values into the 8-bit range.
7. Display the enhanced photograph.
8. Compare the contrast and detail visibility.

6.4 Advantages

1. Provides direct control over intensity ranges.

2. Improves contrast in selected regions.
3. Can enhance faded details.
4. Allows flexible adjustment of dark, middle and bright regions.
5. Can be adjusted according to the photograph.

6.5 Limitations

1. Requires suitable control-point selection.
2. Incorrect points may produce unnatural contrast.
3. May cause clipping if the output range is poorly selected.
4. Does not remove scratches or physical damage.
5. Different photographs may require different control points.

7. PYTHON IMPLEMENTATION USING OPENCV

The following program implements Negative Transformation, Log Transformation, Gamma Transformation and Piecewise-Linear Contrast Stretching.

All techniques are applied to the same input photograph so that their results can be compared fairly.

7.1 Required Libraries

- Python 3
- OpenCV
- NumPy
- Matplotlib
- Time

7.2 Complete Python Program

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import time

image = cv2.imread(
    "old_photo.jpg",
    cv2.IMREAD_GRAYSCALE
)

if image is None:
    raise FileNotFoundError("Image not found")
```

```
# Negative Transformation
start = time.perf_counter()

negative = 255 - image

negative_time = time.perf_counter() - start

# Log Transformation
start = time.perf_counter()

c = 255 / np.log(256)

log_image = c * np.log1p(
    image.astype(np.float32)
)

log_image = np.clip(
    log_image, 0, 255
).astype(np.uint8)

log_time = time.perf_counter() - start

# Gamma Transformation
gamma = 0.6

start = time.perf_counter()

normalized = image.astype(
    np.float32
) / 255.0

gamma_image = 255 * (
    normalized ** gamma
)
```

```
gamma_image = np.clip(
    gamma_image, 0, 255
).astype(np.uint8)

gamma_time = time.perf_counter() - start

# Piecewise-Linear Contrast Stretching
start = time.perf_counter()

r1, r2 = 60, 180
s1, s2 = 30, 225

piecewise = np.interp(
    image,
    [0, r1, r2, 255],
    [0, s1, s2, 255]
 ).astype(np.uint8)

piecewise_time = time.perf_counter() - start

# Display results
images = [
    image,
    negative,
    log_image,
    gamma_image,
    piecewise
]

titles = [
    "Original Image",
    "Negative Transformation",
    "Log Transformation",
    "Gamma Transformation",
    "Piecewise Contrast Stretching"
```

```
]

plt.figure(figsize=(14, 9))

for i in range(len(images)):
    plt.subplot(2, 3, i + 1)
    plt.imshow(images[i], cmap="gray")
    plt.title(titles[i])
    plt.axis("off")

plt.tight_layout()
plt.show()

# Display histograms
plt.figure(figsize=(14, 9))

for i in range(len(images)):
    plt.subplot(2, 3, i + 1)

    plt.hist(
        images[i].ravel(),
        bins=256,
        range=(0, 256)
    )

    plt.title(titles[i] + " Histogram")
    plt.xlabel("Intensity")
    plt.ylabel("Frequency")

plt.tight_layout()
plt.show()

# Processing time
print("Negative:", negative_time)
print("Log:", log_time)
```

```
print("Gamma:", gamma_time)
print("Piecewise:", piecewise_time)
```

8. COMPARISON OF SUBJECT-TO-BACKGROUND CONTRAST

Subject-to-background contrast refers to the difference in brightness between the important subject and the surrounding background.

In an old portrait photograph, the subject may be a person's face or body, while the background may contain walls, furniture or other objects.

When the difference between subject and background intensity is small, the photograph appears faded and important details become difficult to observe.

Grey level transformations can improve this difference.

8.1 Contrast Measurement

A simple subject-to-background contrast measurement can be calculated using:

$C = |\mu s - \mu b|$

Where:

- C = Subject-to-background contrast.
- μs = Mean intensity of the selected subject region.
- μb = Mean intensity of the selected background region.

The subject and background regions should be selected consistently in the original and transformed images.

A higher value indicates a greater difference between the average subject and background intensities. However, this measurement alone does not guarantee better image quality.

8.2 Comparison Table

Technique	Expected Contrast Behaviour
Negative Transformation	Reverses the original intensity relationship
Log Transformation	Expands darker values and compresses brighter values
Gamma Transformation	Changes brightness according to gamma value
Piecewise-Linear Stretching	Expands selected intensity ranges

8.3 Analysis

Negative transformation reverses the appearance of the photograph. It may help reveal certain details but is generally not suitable as the final restored image.

Log transformation improves visibility in darker regions but may compress bright intensity differences.

Gamma transformation provides flexible brightness correction. A gamma value below 1 can improve dark photographs, while a value above 1 can darken excessively bright photographs.

Piecewise-linear contrast stretching provides control over selected intensity ranges. It can improve the contrast between the subject and background when appropriate control points are selected.

9. RECOVERY OF FAINT IMAGE DETAILS

Old photographs may contain faint details due to fading, ageing and uneven illumination.
These details may include:

1. Facial outlines.
2. Eyes and eyebrows.
3. Hair patterns.
4. Clothing texture.
5. Background objects.
6. Handwritten information.

Grey level transformations can improve the visibility of faint details by modifying their intensity values.

9.1 Negative Transformation

Negative transformation reverses pixel intensity values. Certain details that are difficult to observe in the original photograph may become easier to inspect in the negative image.
However, the overall visual appearance is reversed.

9.2 Log Transformation

Log transformation expands darker intensity values. It may improve the visibility of faint information located in darker portions of the photograph.
However, noise in dark regions may also become more noticeable.

9.3 Gamma Transformation

Gamma transformation allows brightness adjustment according to the selected gamma value.
A gamma value below 1 can reveal details in dark photographs.
A gamma value above 1 can reduce excessive brightness.

9.4 Piecewise-Linear Contrast Stretching

Piecewise-linear transformation allows specific intensity ranges to be expanded.
For example, if the important facial details are concentrated in a narrow intensity range, that range can be expanded to improve visibility.

9.5 Comparison

Technique	Faint Detail Recovery
Negative	May reveal details through intensity reversal
Log	Useful for darker details

Gamma	Useful for brightness correction
Piecewise	Useful for selected intensity ranges

10. CLIPPING EFFECTS

Clipping occurs when transformed pixel intensity values exceed the allowed intensity range.

For an 8-bit grayscale image, the valid intensity range is 0 to 255.

Values below 0 are clipped to 0, and values above 255 are clipped to 255.

Clipping can cause the loss of subtle image details.

10.1 Effects of Clipping

1. Very dark regions may become completely black.
2. Very bright regions may become completely white.
3. Fine intensity differences may disappear.
4. Facial details may be lost.
5. The photograph may appear unnatural.

10.2 Clipping Analysis

Negative transformation normally reverses the intensity range without requiring additional clipping.

Log transformation uses normalization to keep values within the valid range.

Gamma transformation may produce values within the 0–255 range when correctly normalized.

Piecewise-linear stretching may cause clipping if the control points are selected incorrectly.

10.3 Python Code to Measure Clipping

```
def clipping_percentage(img):

    total = img.size

    black_pixels = np.count_nonzero(
        img == 0
    )

    white_pixels = np.count_nonzero(
        img == 255
    )

    black_percentage = (
        black_pixels / total
    ) * 100
```

```
    white_percentage = (
        white_pixels / total
    ) * 100

    return black_percentage, white_percentage



results = [
    ("Original", image),
    ("Negative", negative),
    ("Log", log_image),
    ("Gamma", gamma_image),
    ("Piecewise", piecewise)
]

for name, img in results:

    black, white = clipping_percentage(img)

    print(name)
    print("Black clipping:", black, "%")
    print("White clipping:", white, "%")
```

This program measures the percentage of output pixels at the minimum and maximum intensity values. These measurements help identify possible loss of details at the intensity extremes.

11. TRANSFORMATION CURVES

A transformation curve represents the relationship between input intensity and output intensity. The horizontal axis represents the input intensity, while the vertical axis represents the output intensity.

Transformation curves help understand how different techniques modify image brightness and contrast.

11.1 Negative Transformation Curve

Formula:

s = 255 - r

The curve is a straight decreasing line.

When the input intensity increases, the output intensity decreases.

11.2 Log Transformation Curve

Formula:

$s = [255 / \log(256)] \times \log(1 + r)$

The curve rises quickly for lower input intensities and gradually becomes flatter for higher intensities.

It expands darker values and compresses brighter values.

11.3 Gamma Transformation Curve

Formula:

$s = 255 \times (r / 255)^{\gamma}$

The curve depends on the gamma value.

- Gamma below 1 produces a curve above the identity line.
- Gamma equal to 1 produces the identity line.
- Gamma above 1 produces a curve below the identity line.

11.4 Piecewise-Linear Transformation Curve

The curve contains multiple straight-line sections based on selected control points.

It allows different input intensity ranges to receive different levels of contrast enhancement.

11.5 Python Code for Transformation Curves

```
import numpy as np
import matplotlib.pyplot as plt

r = np.arange(256)

negative_curve = 255 - r

c = 255 / np.log(256)

log_curve = c * np.log1p(r)

gamma = 0.6

gamma_curve = 255 * (
    (r / 255) ** gamma
)
```

```
r1, r2 = 60, 180
s1, s2 = 30, 225

piecewise_curve = np.interp(
    r,
    [0, r1, r2, 255],
    [0, s1, s2, 255]
)

plt.figure(figsize=(10, 7))

plt.plot(r, negative_curve, label="Negative")
plt.plot(r, log_curve, label="Log")
plt.plot(r, gamma_curve, label="Gamma")
plt.plot(r, piecewise_curve, label="Piecewise")
plt.plot(r, r, label="Identity")

plt.xlabel("Input Intensity")
plt.ylabel("Output Intensity")
plt.title("Grey Level Transformation Curves")

plt.legend()
plt.grid()

plt.show()
```

12. INPUT AND OUTPUT HISTOGRAM ANALYSIS

A histogram shows the distribution of pixel intensities in an image.

The original histogram helps identify the initial brightness and contrast conditions.

After transformation, the output histogram indicates how the pixel intensity distribution has changed.

12.1 Original Histogram

The original photograph may contain a narrow intensity distribution due to fading and low contrast.

Dark and bright regions may not be clearly separated.

12.2 Negative Histogram

Negative transformation reverses the intensity distribution.

If the original image contains intensity r, its corresponding negative intensity is 255 - r.

12.3 Log Histogram

Log transformation expands darker intensity values and compresses brighter intensity values.

The output histogram may show a redistribution towards brighter intensity values.

12.4 Gamma Histogram

Gamma transformation changes the intensity distribution according to the selected gamma value.

For gamma below 1, many input values move towards brighter output values.

For gamma above 1, many input values move towards darker output values.

12.5 Piecewise Histogram

Piecewise-linear stretching modifies selected intensity ranges.

The output histogram may show an expanded distribution across the selected output ranges.

12.6 Histogram Comparison Code

```
images = [
    image,
    negative,
    log_image,
    gamma_image,
    piecewise
]

titles = [
    "Original",
    "Negative",
    "Log",
    "Gamma",
    "Piecewise"
]

plt.figure(figsize=(14, 9))

for i in range(len(images)):

    plt.subplot(2, 3, i + 1)
```

```
    plt.hist(
        images[i].ravel(),
        bins=256,
        range=(0, 256)
    )

    plt.title(titles[i])
    plt.xlabel("Pixel Intensity")
    plt.ylabel("Frequency")

plt.tight_layout()
plt.show()
```

13. BIT-PLANE SLICING

13.1 Definition

Bit-plane slicing is an image processing technique used to separate an image into individual binary planes.

An 8-bit grayscale image contains eight bit planes, ranging from bit 0 to bit 7.

Each pixel is represented using eight binary bits.

For example:

The decimal value 150 is represented as:

10010110

Each binary position represents a different bit plane.

13.2 Bit-Plane Structure

Bit Plane	Description
Bit 7	Most Significant Bit (MSB)
Bit 6	High-order information
Bit 5	High-order information
Bit 4	Middle-order information
Bit 3	Middle-order information
Bit 2	Low-order information
Bit 1	Low-order information
Bit 0	Least Significant Bit (LSB)

Higher-order bit planes generally contain major image structures and large intensity variations.

Lower-order bit planes often contain fine intensity changes, subtle texture, noise and compression artefacts.

However, useful fine details can also appear in lower-order bit planes.

13.3 Importance in Old Photograph Restoration

Bit-plane slicing helps understand how image information is distributed across different binary levels.

It can help identify:

1. Major subject structures.
2. Facial outlines.
3. Background intensity variations.
4. Fine texture information.
5. Noise and scanner-related artefacts.

Bit-plane slicing does not automatically distinguish damage from useful information. Each plane must be examined visually.

13.4 Python Program for Bit-Plane Slicing

```
import cv2
import matplotlib.pyplot as plt

image = cv2.imread(
    "old_photo.jpg",
    cv2.IMREAD_GRAYSCALE
)

if image is None:
    raise FileNotFoundError("Image not found")

plt.figure(figsize=(12, 8))

for bit in range(8):

    plane = (image >> bit) & 1

    plt.subplot(2, 4, bit + 1)

    plt.imshow(
```

```
        plane * 255,
        cmap="gray"
    )

    plt.title("Bit Plane " + str(bit))
    plt.axis("off")

plt.tight_layout()
plt.show()
```

13.5 Analysis of Bit Planes

Bit Plane 7:

The highest-order bit contributes strongly to the major intensity structure of the image. It may show large dark and bright regions.

Bit Plane 6:

This plane contributes to the broad tonal structure and may show important subject-background information.

Bit Planes 5 and 4:

These planes contain additional intensity information and may help represent facial outlines and medium-scale image details.

Bit Planes 3 and 2:

These planes contain finer intensity variations and may represent texture and subtle image details.

Bit Planes 1 and 0:

These planes contain the smallest intensity variations. They may contain fine details, scanner noise, compression artefacts and small image variations.

13.6 Useful and Noisy Bit Planes

The higher-order bit planes generally contain more important structural information.

The lower-order bit planes may contain a greater amount of noise and small intensity variations. However, it is not correct to assume that all lower-order planes contain only noise. Some useful fine details may also be present.

Therefore, the most useful bit planes must be identified by examining the actual photograph.

14. EFFECT OF APPLYING THE SAME GAMMA VALUE TO BRIGHT AND DARK PHOTOGRAPHS

Gamma transformation changes the brightness of an image according to the selected gamma value.

Applying the same gamma value to different photographs may produce different visual results because the original images have different intensity distributions.

14.1 Dark Photograph

A dark photograph contains a large number of low-intensity pixels.

When gamma is less than 1, the lower intensity values are expanded towards brighter values.

This may improve the visibility of hidden details.

However, excessive brightening can also increase noise visibility.

14.2 Bright Photograph

A bright photograph contains a larger number of high-intensity pixels.

Applying the same gamma value below 1 may make the photograph excessively bright.

Some bright regions may lose their natural appearance.

A gamma value greater than 1 may be more appropriate for reducing excessive brightness.

14.3 Comparison Table

Image Type	Gamma Value	Expected Result
Dark photograph	0.5	Brightness increases
Dark photograph	1.0	No significant change
Dark photograph	2.0	Image becomes darker
Bright photograph	0.5	May become excessively bright
Bright photograph	1.0	No significant change
Bright photograph	2.0	Brightness decreases

14.4 Python Program for Gamma Comparison

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

dark_image = cv2.imread(
    "dark_photo.jpg",
    cv2.IMREAD_GRAYSCALE
)

bright_image = cv2.imread(
    "bright_photo.jpg",
    cv2.IMREAD_GRAYSCALE
)
```

```
if dark_image is None or bright_image is None:
    raise FileNotFoundError("Input photograph not found")

gamma = 0.6

def gamma_correction(img, gamma):

    normalized = img.astype(
        np.float32
    ) / 255.0

    corrected = 255 * (
        normalized ** gamma
    )

    return np.clip(
        corrected, 0, 255
    ).astype(np.uint8)

dark_result = gamma_correction(
    dark_image, gamma
)

bright_result = gamma_correction(
    bright_image, gamma
)

plt.figure(figsize=(10, 7))

plt.subplot(2, 2, 1)
plt.imshow(dark_image, cmap="gray")
plt.title("Original Dark Image")
plt.axis("off")
```

```
plt.subplot(2, 2, 2)
plt.imshow(dark_result, cmap="gray")
plt.title("Dark Image After Gamma")
plt.axis("off")

plt.subplot(2, 2, 3)
plt.imshow(bright_image, cmap="gray")
plt.title("Original Bright Image")
plt.axis("off")

plt.subplot(2, 2, 4)
plt.imshow(bright_result, cmap="gray")
plt.title("Bright Image After Gamma")
plt.axis("off")

plt.tight_layout()
plt.show()
```

14.5 Analysis

The same gamma value does not guarantee the same level of enhancement for every photograph. A dark photograph may benefit from a gamma value below 1, while a bright photograph may require a gamma value greater than 1.

Therefore, gamma correction should be selected according to the original brightness distribution of each photograph.

An adaptive approach that selects gamma based on image brightness may provide more consistent results.

15. COMPARISON WITH A PRETRAINED LOW-LIGHT OR IMAGE-ENHANCEMENT MODEL

15.1 Introduction

Traditional grey level transformations modify pixel intensities using predefined mathematical equations.

Pretrained image-enhancement models use deep learning techniques to learn image enhancement patterns from training data.

These models may improve brightness, contrast and visibility in difficult lighting conditions.

A representative low-light enhancement model is Zero-DCE, which learns to enhance low-light images using image-specific enhancement curves.

Other image restoration models may be used for different enhancement tasks.

15.2 Traditional Transformation Methods

Traditional techniques include:

1. Negative Transformation.
2. Log Transformation.
3. Gamma Transformation.
4. Piecewise-Linear Contrast Stretching.

These methods are relatively simple and computationally efficient.

They do not require training data or a GPU for basic implementation.

15.3 Pretrained Enhancement Model

A pretrained enhancement model uses learned parameters to transform an input photograph.

Depending on the model, it may improve:

1. Low-light visibility.
2. Brightness distribution.
3. Local contrast.
4. Appearance of dark regions.
5. Overall visual quality.

However, a pretrained model may produce unwanted changes or introduce visual artefacts.

It must be checked carefully to ensure that original facial features and historical details are preserved.

15.4 Comparison Table

Evaluation Parameter	Traditional Transformation	Pretrained Enhancement Model
Implementation	Simple mathematical functions	Deep learning model
Training requirement	Not required	Model training is performed beforehand
Processing resources	Generally low	May require more resources
Brightness adjustment	Formula-based	Learned enhancement
Local enhancement	Limited without additional methods	Depends on model
Noise handling	Not designed specifically for noise removal	Depends on model
Historical detail preservation	Easier to inspect and control	Requires careful verification

Hardware requirement	Can run on CPU	Depends on model size and implementation

15.5 Limitations of Pretrained Models

1. May introduce unwanted visual changes.
2. May not perform equally well on every old photograph.
3. Can require more computational resources.
4. May produce artefacts in damaged regions.
5. May not preserve every original detail.

A pretrained enhancement model should not be assumed to reconstruct missing historical information accurately.

If a model generates or modifies details, the result must be labelled as enhanced or reconstructed rather than treated as an exact recovery of the original photograph.

15.6 Model Comparison Procedure

1. Select the same original photographs for both approaches.
2. Apply the selected traditional transformation.
3. Process the same photographs using a pretrained enhancement model.
4. Compare brightness and contrast.
5. Examine facial details and background preservation.
6. Check noise and unwanted artefacts.
7. Measure processing time.
8. Record the results in a comparison table.

If a pretrained model cannot be executed in the available environment, its comparison should be reported as a reasoned technical discussion rather than as an actual experiment.

16. OVERALL COMPARISON OF THE THREE CANDIDATE TECHNIQUES

The three candidate techniques are compared based on their performance in old photograph restoration.

Evaluation Parameter	**Negative and Log**	**Gamma Transformation**	**Piecewise-Linear Stretching**
Brightness correction	Log expands dark values; negative reverses intensity	Flexible brightness adjustment	Depends on control points
Contrast improvement	Depends on image and transformation	Depends on gamma value	Selected intensity ranges can be expanded

Faint detail visibility	Log may improve dark details	Can improve details through brightness correction	Can enhance selected intensity ranges
Noise visibility	Log may reveal dark-region noise	Excessive brightening may reveal noise	Strong stretching may reveal noise
Clipping risk	Depends on implementation and normalization	Possible with poor scaling or later processing	Depends on control-point selection
Parameter selection	Low for negative and log	Gamma value required	Control points required
Processing complexity	Low	Low	Low

17. EXPERIMENTAL RESULT TABLE

The following table should be completed after running the Python programs on the selected photographs.

Technique	Subject-Background Contrast	Faint Detail Visibility	Clipping Observation	Processing Time
Negative	To be measured	To be observed	To be measured	To be measured
Log	To be measured	To be observed	To be measured	To be measured
Gamma	To be measured	To be observed	To be measured	To be measured
Piecewise	To be measured	To be observed	To be measured	To be measured

The comparison must use the same photographs and hardware for all techniques.

Actual measurements should be recorded instead of entering assumed values.

18. ADVANTAGES OF GREY LEVEL TRANSFORMATION

1. Simple implementation using Python and OpenCV.
2. Improves the visibility of faded photographs.
3. Supports brightness and contrast correction.
4. Helps reveal details in dark regions.
5. Requires relatively low computational resources.
6. Provides multiple techniques for different image conditions.

7. Can be used as a preprocessing stage in a restoration pipeline.

19. LIMITATIONS OF GREY LEVEL TRANSFORMATION

1. Does not remove scratches or dust.
2. Cannot restore physically missing image regions.
3. May amplify existing noise.
4. Excessive enhancement may cause clipping.
5. A single transformation may not work for every photograph.
6. May change the natural appearance of the original image.
7. Does not guarantee preservation of all historical details.

20. APPLICATIONS

Grey level transformations are useful in several image processing applications.

1. Old Photograph Restoration: Improves the visibility of faded photographs.

2. Medical Image Processing: Helps improve the visibility of selected structures in grayscale medical images.

3. Satellite Image Processing: Enhances intensity differences in remote-sensing images.

4. Document Image Processing: Helps reveal faint text and printed information.

5. Industrial Inspection: Improves the visibility of defects and surface variations.

6. Digital Archiving: Supports the enhancement and examination of historical photographs.

21. FINAL TECHNIQUE SELECTION

Among the three candidate techniques, Piecewise-Linear Contrast Stretching is selected as a suitable primary candidate for further evaluation in the old photograph restoration system. The reason is that it provides direct control over selected intensity ranges. This is useful when the important subject details occupy a narrow range of grayscale values.

Gamma Transformation is also useful for correcting photographs that are generally too dark or too bright.

Log Transformation can help reveal information in darker regions, while Negative Transformation is mainly useful for visual analysis rather than producing the final natural-looking photograph.

The final technique should be selected after examining the actual restored images, subject-to-background contrast, clipping percentage, faint detail visibility and processing time.

For photographs with different brightness conditions, the transformation parameters should be adjusted individually rather than applying one fixed setting to every image.

22. CONCLUSION

Grey Level Transformation is an important technique in digital image processing that helps improve the brightness, contrast and visibility of old photographs.

In this task, three candidate techniques were studied and implemented using Python and OpenCV. These techniques are Negative and Log Transformation, Power-Law or Gamma Transformation, and Piecewise-Linear Contrast Stretching.

Negative transformation reverses the intensity values of the image, while logarithmic transformation expands darker intensity values. Gamma transformation provides flexible brightness correction according to the selected gamma value. Piecewise-linear contrast stretching allows selected intensity ranges to be expanded to improve contrast.

Transformation curves and input/output histograms help explain the behaviour of each technique. Bit-plane slicing provides additional information about how image details, texture and noise are distributed across the binary planes.

The comparison also shows that applying the same gamma value to different photographs may produce different results because each photograph has a different brightness distribution. Pretrained image-enhancement models can provide more advanced enhancement, but their output must be checked carefully to avoid unwanted changes to original facial features and historical details.

Therefore, grey level transformation provides a useful foundation for old photograph restoration. The most suitable method should be selected according to the photograph's condition, measured contrast, detail preservation and processing requirements.

TASK 2 – HISTOGRAM PROCESSING AND ENHANCEMENT

1. INTRODUCTION

Histogram processing is an important image enhancement technique used to improve the brightness and contrast of digital images.

Old photographs may contain faded regions, uneven illumination, dark areas and low contrast due to ageing. These problems make facial features and background details difficult to observe. Histogram-based techniques improve image quality by modifying the distribution of pixel intensity values.

In this task, three techniques are implemented and compared:

1. Global Histogram Equalization.
2. Histogram Matching.
3. Contrast Limited Adaptive Histogram Equalization (CLAHE).

2. OBJECTIVES

1. To understand histogram processing.
2. To implement three histogram enhancement techniques.
3. To improve contrast and detail visibility.
4. To generate histograms and CDF plots.
5. To analyse noise amplification.
6. To compare different CLAHE tile sizes and clip limits.
7. To select a suitable technique for old photograph restoration.

3. HISTOGRAM FUNDAMENTALS

A histogram represents the distribution of pixel intensity values in an image.

For an 8-bit grayscale image, intensity values range from 0 to 255.

- 0 represents black.
- 255 represents white.
- Intermediate values represent shades of grey.

The histogram is represented as:

$H(rk) = nk$

Where:

- rk = Intensity level.
- nk = Number of pixels having that intensity.

The normalized histogram is:

$P(rk) = nk / (M \times N)$

Where M and N represent the image dimensions.

4. TECHNIQUE A – GLOBAL HISTOGRAM EQUALIZATION

4.1 Definition

Global Histogram Equalization improves image contrast by redistributing pixel intensity values across the available grayscale range.

It uses the cumulative distribution function (CDF) to calculate new intensity values.

4.2 Mathematical Formula

$S = (L - 1) \times CDF(r)$

Where:

- S = Output intensity.
- L = Number of intensity levels.
- CDF(r) = Cumulative distribution function.

For an 8-bit image:

$S = 255 \times CDF(r)$

4.3 Working Procedure

1. Load the grayscale image.
2. Calculate its histogram.
3. Calculate the cumulative distribution function.
4. Generate the intensity mapping.
5. Apply the mapping to the image.
6. Display and compare the enhanced result.

4.4 Advantages

- Improves overall image contrast.
- Simple to implement.
- Requires low computational resources.
- Can reveal hidden image details.

4.5 Limitations

- Does not improve every region equally.
- May amplify noise.
- Can produce unnatural brightness.
- May not work well with uneven illumination.

5. TECHNIQUE B – HISTOGRAM MATCHING

5.1 Definition

Histogram Matching, also called Histogram Specification, modifies the intensity distribution of a source image to match a reference image.

It is useful when a suitable reference photograph is available.

5.2 Mathematical Formula

$z = Fr^{-1}(Fs(r))$

Where:

- Fs(r) = CDF of the source image.
- Fr(z) = CDF of the reference image.
- r = Source intensity.
- z = Output intensity.

5.3 Working Procedure

1. Load the source photograph.
2. Load a suitable reference photograph.
3. Calculate both histograms.
4. Calculate their CDF values.
5. Generate an intensity mapping.

6. Apply the mapping to the source image.
7. Display the matched result.

5.4 Advantages

- Provides reference-based contrast adjustment.
- Helps standardize photographs.
- Gives control over the target intensity distribution.

5.5 Limitations

- Requires a suitable reference image.
- Poor reference selection may produce unnatural results.
- Does not remove physical damage.
- Matching histograms does not guarantee detail preservation.

6. TECHNIQUE C – CLAHE

6.1 Definition

Contrast Limited Adaptive Histogram Equalization (CLAHE) improves local image contrast by dividing an image into smaller regions called tiles.

Each tile is processed separately, and a clipping limit is applied to control excessive contrast amplification.

It is useful for old photographs containing uneven illumination and faded regions.

6.2 Working Procedure

1. Divide the image into smaller tiles.
2. Calculate the histogram of each tile.
3. Apply a clipping limit.
4. Redistribute clipped histogram values.
5. Perform local histogram equalization.
6. Combine the regions using interpolation.
7. Generate the enhanced image.

6.3 Important Parameters

Tile Grid Size: Determines how the image is divided into regions. Smaller tiles provide more local enhancement.

Clip Limit: Controls contrast amplification. A lower value reduces excessive enhancement, while a higher value produces stronger contrast.

6.4 Advantages

- Improves local contrast.
- Handles uneven illumination.
- Helps reveal faded details.

- Controls excessive enhancement through clipping.

6.5 Limitations

- Requires parameter selection.
- Small tiles may increase noise visibility.
- High clip limits may produce unnatural results.

7. PYTHON IMPLEMENTATION

The following program implements all three techniques using the same input photograph.

7.1 Required Libraries

Python, OpenCV, NumPy, Matplotlib and Time.

7.2 Complete Python Program

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import time

image = cv2.imread(
    "old_photo.jpg",
    cv2.IMREAD_GRAYSCALE
)

reference = cv2.imread(
    "reference_photo.jpg",
    cv2.IMREAD_GRAYSCALE
)

if image is None or reference is None:
    raise FileNotFoundError("Input image not found")

# Global Histogram Equalization
start = time.perf_counter()

global_equalized = cv2.equalizeHist(image)

global_time = time.perf_counter() - start
```

```
# Histogram Matching
start = time.perf_counter()

source_hist, _ = np.histogram(
    image.ravel(), 256, [0, 256]
)

reference_hist, _ = np.histogram(
    reference.ravel(), 256, [0, 256]
)

source_cdf = np.cumsum(source_hist)
reference_cdf = np.cumsum(reference_hist)

source_cdf = source_cdf / source_cdf[-1]
reference_cdf = reference_cdf / reference_cdf[-1]

mapping = np.interp(
    source_cdf,
    reference_cdf,
    np.arange(256)
)

matched = mapping[image].astype(np.uint8)

matching_time = time.perf_counter() - start

# CLAHE
start = time.perf_counter()

clahe = cv2.createCLAHE(
    clipLimit=2.0,
    tileGridSize=(8, 8)
)
```

```
clahe_image = clahe.apply(image)

clahe_time = time.perf_counter() - start

# Display results
images = [
    image,
    global_equalized,
    matched,
    clahe_image
]

titles = [
    "Original",
    "Global  Equalization",
    "Histogram Matching",
    "CLAHE"
]

plt.figure(figsize=(12, 8))

for i in range(4):
    plt.subplot(2, 2, i + 1)
    plt.imshow(images[i], cmap="gray")
    plt.title(titles[i])
    plt.axis("off")

plt.tight_layout()
plt.show()

# Display histograms
plt.figure(figsize=(12, 8))

for i in range(4):
    plt.subplot(2, 2, i + 1)
```

```
    plt.hist(
        images[i].ravel(),
        bins=256,
        range=(0, 256)
    )
    plt.title(titles[i] + " Histogram")

plt.tight_layout()
plt.show()

print("Global Equalization:", global_time)
print("Histogram Matching:", matching_time)
print("CLAHE:", clahe_time)
```

8. HISTOGRAM AND CDF ANALYSIS

8.1 Histogram Analysis

The original histogram shows the brightness distribution of the photograph.

After enhancement:

- Global Equalization redistributes overall intensity values.
- Histogram Matching adjusts the distribution according to the reference image.
- CLAHE improves contrast in individual regions.

8.2 Cumulative Distribution Function

CDF represents the cumulative probability of intensity values.

$CDF(k) = \Sigma P(rj)$

Where P(rj) is the normalized histogram probability.

CDF is used to calculate the intensity mapping in histogram equalization.

8.3 Python Code for CDF Plot

```
import numpy as np
import matplotlib.pyplot as plt

images = [
    image,
    global_equalized,
    matched,
    clahe_image
]
```

```
titles = [
    "Original",
    "Global  Equalization",
    "Histogram Matching",
    "CLAHE"
]

plt.figure(figsize=(12, 8))

for i in range(4):

    hist, _ = np.histogram(
        images[i].ravel(),
        bins=256,
        range=(0, 256)
    )

    cdf = hist.cumsum()
    cdf = cdf / cdf[-1]

    plt.subplot(2, 2, i + 1)
    plt.plot(cdf)
    plt.title(titles[i] + " CDF")
    plt.xlabel("Intensity")
    plt.ylabel("Cumulative Probability")

plt.tight_layout()
plt.show()
```

9. WHY GLOBAL HISTOGRAM EQUALIZATION MAY FAIL

Global Histogram Equalization applies one transformation to the entire image.

Old photographs may contain both dark and bright regions. A single transformation may not improve all regions equally.

Major problems include:

1. Uneven illumination may remain.

2. Noise in faded regions may become more visible.
3. Bright areas may become over-enhanced.
4. Facial details may lose natural appearance.
5. Small local details may not receive sufficient enhancement.

Therefore, CLAHE can be more suitable for images containing local brightness variations.

10. CLAHE PARAMETER COMPARISON

CLAHE performance depends on tile size and clip limit.

10.1 Tile Grid Size

Tile Size	Expected Effect
4 × 4	Stronger local enhancement
8 × 8	Balanced enhancement
16 × 16	Enhancement over larger regions

10.2 Clip Limit

Clip Limit	Expected Effect
1.0	Lower contrast amplification
2.0	Moderate enhancement
4.0	Stronger enhancement

10.3 Python Code

```
import cv2
import matplotlib.pyplot as plt

settings = [
   (1.0, (4, 4)),
   (2.0, (8, 8)),
   (4.0, (8, 8)),
   (2.0, (16, 16))
]

plt.figure(figsize=(12, 8))

for i, (clip, tile) in enumerate(settings):

   clahe = cv2.createCLAHE(
      clipLimit=clip,
```

```
        tileGridSize=tile
    )

    result = clahe.apply(image)

    plt.subplot(2, 2, i + 1)
    plt.imshow(result, cmap="gray")
    plt.title(
        "Clip: " + str(clip) +
        " Tile: " + str(tile)
    )
    plt.axis("off")

plt.tight_layout()
plt.show()
```

A moderate clip limit and suitable tile size should be selected after examining noise, facial details and local contrast.

11. EFFECT OF GLOBAL EQUALIZATION ON A UNIFORM BACKGROUND

A photograph may contain a large area with nearly uniform background intensity.

This background can contribute significantly to the global histogram.

As a result:

1. Small background variations may become visible.
2. Noise may be amplified.
3. Subject and background enhancement may become unbalanced.
4. The final image may appear unnatural.

CLAHE provides local enhancement, but its parameters must also be selected carefully.

12. COMPARISON OF THREE TECHNIQUES

Parameter	**Global Equalization**	**Histogram Matching**	**CLAHE**
Contrast	Global	Reference-based	Local
Detail visibility	Overall	Depends on reference	Local
Noise amplification	Possible	Possible	Controlled through clipping
Uneven illumination	Limited	Reference-dependent	Better local adaptation
Parameters	Few	Reference required	Clip limit and tile size
Complexity	Low	Moderate	Moderate

Processing Time

Technique	Processing Time
Global Equalization	To be measured
Histogram Matching	To be measured
CLAHE	To be measured

Actual processing times should be recorded after executing the program on the same computer and image.

13. FINAL TECHNIQUE SELECTION

CLAHE is selected as a suitable candidate for old photographs containing uneven illumination and local contrast variations.

Global Histogram Equalization is useful for improving overall contrast, while Histogram Matching is useful when a suitable reference photograph is available.

CLAHE provides local contrast enhancement and limits excessive amplification through clipping.

The final selection should be confirmed by comparing actual images, histograms, CDF plots, noise visibility and processing time.

14. CONCLUSION

Histogram processing is an important technique for improving the visual quality of faded and low-contrast photographs.

In this task, Global Histogram Equalization, Histogram Matching and CLAHE were studied and implemented using Python and OpenCV.

Global equalization improves overall contrast, Histogram Matching adjusts the image according to a reference distribution, and CLAHE improves local contrast.

The comparison shows that CLAHE is a suitable candidate for photographs containing uneven illumination because it provides local enhancement with controlled contrast amplification.

Therefore, histogram processing can improve the visibility of important details in old photographs. However, the original facial features and historical appearance must be preserved during enhancement.

TASK 3 – SPATIAL FILTERING AND IMAGE SMOOTHING

1. INTRODUCTION

Spatial filtering is an image processing technique used to reduce noise and improve image quality by modifying pixel values based on their neighbouring pixels.

Old photographs may contain Gaussian noise, salt-and-pepper noise, dust particles and unwanted marks. These problems reduce image clarity and affect important details such as eyes, hair, facial features and clothing texture.

Spatial filtering helps reduce unwanted noise while preserving important image information.

In this task, three filtering techniques are implemented:

1. Box Filtering.
2. Gaussian Filtering.
3. Median Filtering.

Different kernel sizes are tested to compare noise removal, detail preservation and processing time.

2. OBJECTIVES

1. To understand spatial filtering and image smoothing.
2. To implement Box, Gaussian and Median filters.
3. To compare 3×3, 5×5 and 7×7 kernels.
4. To analyse noise reduction and detail preservation.
5. To calculate PSNR and SSIM when a clean reference is available.
6. To study image intensity profiles.
7. To understand correlation and convolution.
8. To compare traditional filtering with a pretrained DnCNN model.

3. SPATIAL FILTERING FUNDAMENTALS

Spatial filtering processes an image using a small matrix called a kernel or mask.

The kernel moves across the image and calculates new pixel values using neighbouring pixels.

The general filtering operation is:

$g(x,y) = \Sigma\Sigma\, w(i,j)\, f(x-i,y-j)$

Where:

- f(x,y) = Input image.
- g(x,y) = Output image.
- w(i,j) = Filter kernel.
- i,j = Kernel positions.

Spatial filtering is mainly used for noise removal, smoothing and image enhancement.

4. TECHNIQUE A – BOX FILTERING

4.1 Definition

Box Filtering is a linear smoothing technique that replaces each pixel with the average value of its neighbouring pixels.

It uses a kernel in which all elements have equal weights.

4.2 Mathematical Formula

For a 3×3 box filter:

$K = (1/9) \times$

$$\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

The output pixel is calculated by averaging the nine neighbouring pixels.

4.3 Working Procedure

1. Load the original grayscale image.
2. Select the kernel size.
3. Apply the averaging operation.
4. Calculate the average intensity of neighbouring pixels.
5. Replace the centre pixel with the calculated value.
6. Display the filtered image.

4.4 Advantages

- Simple and fast.
- Reduces small intensity variations.
- Easy to implement.
- Requires low computational resources.

4.5 Limitations

- Can blur edges.
- Reduces fine details.
- Not very effective for salt-and-pepper noise.
- Large kernels may remove important facial details.

5. TECHNIQUE B – GAUSSIAN FILTERING

5.1 Definition

Gaussian Filtering is a smoothing technique that reduces image noise using a Gaussian-weighted kernel.

Nearby pixels receive higher weights, while distant pixels receive lower weights.

It provides smoother results than simple averaging in many situations.

5.2 Mathematical Formula

The two-dimensional Gaussian function is:

$G(x,y) = (1 / 2\pi\sigma^2) \times e^{[-(x^2+y^2)/2\sigma^2]}$

Where:

- σ = Standard deviation.
- x,y = Pixel positions.
- e = Exponential constant.

5.3 Working Procedure

1. Load the grayscale image.
2. Select the Gaussian kernel size.
3. Set the standard deviation.
4. Apply Gaussian filtering.
5. Calculate weighted averages of neighbouring pixels.
6. Display the smoothed image.

5.4 Advantages

- Effective for Gaussian noise.
- Produces smooth image results.
- Gives higher importance to nearby pixels.
- Useful as a preprocessing technique.

5.5 Limitations

- May blur edges.
- Can remove fine facial details.
- Large kernels may reduce image sharpness.

6. TECHNIQUE C – MEDIAN FILTERING

6.1 Definition

Median Filtering is a nonlinear smoothing technique that replaces each pixel with the median value of its neighbouring pixels.

It is especially effective for removing salt-and-pepper noise.

Unlike averaging filters, it selects the middle value after sorting the neighbouring pixel intensities.

6.2 Mathematical Principle

For a 3×3 neighbourhood:

1. Collect the nine pixel values.
2. Arrange them in ascending order.
3. Select the middle value.
4. Replace the centre pixel with the median value.

6.3 Working Procedure

1. Load the original grayscale image.
2. Select an odd-sized kernel.
3. Collect neighbouring pixel values.
4. Sort the values.
5. Calculate the median.
6. Replace the centre pixel.
7. Display the filtered image.

6.4 Advantages

- Effective for salt-and-pepper noise.
- Preserves edges better than simple averaging in many cases.
- Removes isolated noisy pixels.
- Useful for old photographs containing small dark and bright noise spots.

6.5 Limitations

- Large kernels may remove fine details.
- May not remove Gaussian noise as effectively as Gaussian filtering.
- Processing can be slower for larger kernels.

7. KERNEL SIZE COMPARISON

Kernel size determines how many neighbouring pixels are used during filtering.

The following kernel sizes are tested:

Kernel Size	Expected Effect
3×3	Light smoothing with better detail preservation
5×5	Moderate smoothing
7×7	Stronger smoothing with greater risk of detail loss

A small kernel preserves more image information, while a larger kernel removes more noise but may blur important details.

For old photographs, kernel size must be selected carefully to preserve eyes, hair and narrow facial boundaries.

8. PYTHON IMPLEMENTATION

The following program applies Box, Gaussian and Median filtering using 3×3, 5×5 and 7×7 kernels.

8.1 Required Libraries

- Python
- OpenCV
- NumPy
- Matplotlib

- Time

8.2 Complete Python Program

```
import cv2
import matplotlib.pyplot as plt
import time

image = cv2.imread(
    "old_photo.jpg",
    cv2.IMREAD_GRAYSCALE
)

if image is None:
    raise FileNotFoundError("Image not found")

kernel_sizes = [3, 5, 7]

results = []

for k in kernel_sizes:

    # Box Filtering
    start = time.perf_counter()

    box = cv2.blur(image, (k, k))

    box_time = time.perf_counter() - start

    # Gaussian Filtering
    start = time.perf_counter()

    gaussian = cv2.GaussianBlur(
        image, (k, k), 0
    )

    gaussian_time = time.perf_counter() - start
```

```
    # Median Filtering
    start = time.perf_counter()

    median = cv2.medianBlur(image, k)

    median_time = time.perf_counter() - start

    results.append(
        (k, box, gaussian, median)
    )

    print("Kernel Size:", k)
    print("Box Time:", box_time)
    print("Gaussian Time:", gaussian_time)
    print("Median Time:", median_time)

# Display results
plt.figure(figsize=(12, 10))

position = 1

plt.subplot(4, 3, position)
plt.imshow(image, cmap="gray")
plt.title("Original Image")
plt.axis("off")

position += 1

for k, box, gaussian, median in results:

    plt.subplot(4, 3, position)
    plt.imshow(box, cmap="gray")
    plt.title("Box " + str(k) + "x" + str(k))
    plt.axis("off")
```

```
    position += 1

    plt.subplot(4, 3, position)
    plt.imshow(gaussian, cmap="gray")
    plt.title("Gaussian " + str(k) + "x" + str(k))
    plt.axis("off")

    position += 1

    plt.subplot(4, 3, position)
    plt.imshow(median, cmap="gray")
    plt.title("Median " + str(k) + "x" + str(k))
    plt.axis("off")

    position += 1

plt.tight_layout()
plt.show()
```

9. NOISE REDUCTION AND DETAIL PRESERVATION

Different filters perform differently depending on the type of noise present in the photograph.

Box Filtering

Box filtering reduces small intensity variations by averaging neighbouring pixels. However, it may blur facial outlines and fine textures.

Gaussian Filtering

Gaussian filtering is suitable for reducing Gaussian noise. It produces smooth results but may reduce the sharpness of important edges.

Median Filtering

Median filtering is effective for removing salt-and-pepper noise. It can preserve edges better than averaging filters, making it useful for photographs containing isolated dark and bright spots.

Comparison Table

Parameter	Box Filter	Gaussian Filter	Median Filter
Gaussian noise	Moderate	Good	Moderate
Salt-and-pepper noise	Limited	Limited	Good

Edge preservation	Low	Moderate	Better in many cases
Smoothing	Average-based	Weighted	Median-based
Fine detail preservation	May reduce details	May blur details	Depends on kernel size

10. PSNR AND SSIM MEASUREMENT

PSNR and SSIM are commonly used to evaluate image quality.

10.1 Peak Signal-to-Noise Ratio (PSNR)

PSNR measures the difference between a reference image and a processed image.

$MSE = (1/MN) \times \Sigma[I(x,y) - K(x,y)]^2$

$PSNR = 10 \log_{10}(MAX^2 / MSE)$

Where:

- I = Reference image.
- K = Processed image.
- MSE = Mean Squared Error.
- MAX = Maximum pixel value, 255 for an 8-bit image.

A higher PSNR generally indicates lower pixel-wise error.

10.2 Structural Similarity Index (SSIM)

SSIM measures similarity between two images based on luminance, contrast and structural information.

Its value generally ranges from -1 to 1, with 1 representing identical images.

10.3 Python Code

```
import cv2
from skimage.metrics import (
    peak_signal_noise_ratio,
    structural_similarity
)

reference = cv2.imread(
    "clean_reference.jpg",
    cv2.IMREAD_GRAYSCALE
)

filtered = cv2.imread(
    "filtered_image.jpg",
    cv2.IMREAD_GRAYSCALE
)
```

```
if reference is None or filtered is None:
    raise FileNotFoundError("Image not found")

psnr = peak_signal_noise_ratio(
    reference,
    filtered,
    data_range=255
)

ssim = structural_similarity(
    reference,
    filtered,
    data_range=255
)

print("PSNR:", psnr)
print("SSIM:", ssim)
```

A clean reference image is required for meaningful PSNR and SSIM evaluation. Comparing a filtered image directly with a noisy original measures the difference from that noisy image, not necessarily the actual restoration quality.

11. GRAYSCALE INTENSITY PROFILE ANALYSIS

An intensity profile represents pixel brightness values along a selected row or column of an image.

It helps analyse how filtering changes edges, noise and image details.

For example, a profile passing through a face may show changes in intensity between the eyes, nose, skin and background.

Importance

1. Shows variations in pixel intensity.
2. Helps identify noise reduction.
3. Shows whether edges become blurred.
4. Helps compare different kernel sizes.
5. Supports analysis of detail preservation.

Python Code

```
import cv2
```

```
import matplotlib.pyplot as plt

image = cv2.imread(
    "old_photo.jpg",
    cv2.IMREAD_GRAYSCALE
)

box = cv2.blur(image, (5, 5))

gaussian = cv2.GaussianBlur(
    image, (5, 5), 0
)

median = cv2.medianBlur(image, 5)

row = image.shape[0] // 2

plt.figure(figsize=(10, 6))

plt.plot(image[row, :], label="Original")
plt.plot(box[row, :], label="Box")
plt.plot(gaussian[row, :], label="Gaussian")
plt.plot(median[row, :], label="Median")

plt.title("Grayscale Intensity Profiles")
plt.xlabel("Pixel Position")
plt.ylabel("Intensity")

plt.legend()
plt.grid()
plt.show()
```

12. CORRELATION AND CONVOLUTION USING A 3×3 MASK

12.1 Correlation

Correlation is a spatial filtering operation in which the kernel is applied directly to the image without rotating it.

The kernel moves across the image and calculates weighted sums of neighbouring pixels.

12.2 Convolution

Convolution is similar to correlation, but the kernel is rotated by 180 degrees before applying it to the image.

12.3 Main Difference

Correlation	Convolution
Kernel is used directly	Kernel is rotated by 180°
No kernel flipping	Kernel flipping is performed
Used in image matching	Used in many filtering operations

For a symmetric kernel, such as a standard Gaussian or box kernel, correlation and convolution produce the same result.

For an asymmetric kernel, their results may differ.

12.4 Python Code

```
import cv2
import numpy as np

image = cv2.imread(
    "old_photo.jpg",
    cv2.IMREAD_GRAYSCALE
)

kernel = np.array([
    [1, 2, 0],
    [0, 1, 2],
    [2, 0, 1]
], dtype=np.float32)

correlation = cv2.filter2D(
    image, -1, kernel
)

convolution_kernel = cv2.flip(
    kernel, -1
)
```

```
convolution = cv2.filter2D(
    image, -1, convolution_kernel
)

cv2.imshow("Correlation", correlation)
cv2.imshow("Convolution", convolution)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

13. EFFECT OF 7×7 FILTERING ON FACIAL DETAILS

A 7×7 kernel uses 49 neighbouring pixels to calculate each output pixel.

It produces stronger smoothing than 3×3 and 5×5 kernels.

However, large kernels may remove important image details.

Effects on Facial Features

Eyes: Small eye details and eyelashes may become less visible.

Hair: Fine hair patterns may become blurred.

Narrow Boundaries: Thin boundaries around the face and clothing may lose sharpness.

Skin Texture: Small texture variations may disappear.

Therefore, a 7×7 kernel should be used only when stronger noise reduction is necessary. Smaller kernels are often preferable when preserving historical facial details is important.

14. COMPARISON WITH PRETRAINED DnCNN MODEL

DnCNN stands for Denoising Convolutional Neural Network.

It is a deep learning-based image denoising model designed to remove image noise while preserving useful image structures.

Unlike traditional filters, DnCNN learns noise patterns from training images.

Comparison Table

Parameter	Traditional Filters	DnCNN
Method	Mathematical filtering	Deep learning
Noise removal	Depends on filter	Learned noise removal
Gaussian noise	Gaussian filter is useful	Designed for denoising
Salt-and-pepper noise	Median filter is useful	Depends on training and model
Detail preservation	Depends on kernel size	Depends on model performance
Processing resources	Generally low	May require more resources
Implementation	Simple	Requires pretrained model

Limitations of DnCNN

1. Requires a suitable pretrained model.
2. May require more computational resources.
3. Performance depends on training data.
4. May not handle every type of photograph damage.
5. Its output must be checked for loss of fine details.

A pretrained DnCNN model should be evaluated using the same test photographs as the traditional filters. If it cannot be executed, the comparison should be reported as a technical discussion rather than an actual experiment.

15. OVERALL EXPERIMENTAL COMPARISON

Evaluation Parameter	Box	Gaussian	Median
Noise reduction	Moderate	Good for Gaussian noise	Good for impulse noise
Edge preservation	Low	Moderate	Better in many cases
Fine detail	May blur	May blur	Depends on kernel
Processing time	To be measured	To be measured	To be measured
Best use	General smoothing	Gaussian noise	Salt-and-pepper noise

The actual performance should be recorded using the same photographs and hardware.

16. FINAL TECHNIQUE SELECTION

Median filtering is a suitable candidate for photographs containing salt-and-pepper noise because it removes isolated noisy pixels while often preserving edges better than averaging filters.

Gaussian filtering is suitable for reducing Gaussian noise, while Box filtering provides simple general smoothing.

For old photograph restoration, a smaller kernel such as 3×3 or 5×5 should be considered first to avoid losing important facial features.

The final filter should be selected according to noise type, detail preservation, PSNR, SSIM and processing time.

17. CONCLUSION

Spatial filtering is an important technique for reducing noise and improving the quality of old photographs.

In this task, Box Filtering, Gaussian Filtering and Median Filtering were implemented using Python and OpenCV.

Different kernel sizes were compared to understand their effects on noise reduction and image detail preservation.

Box filtering provides simple smoothing, Gaussian filtering is effective for Gaussian noise, and Median filtering is suitable for salt-and-pepper noise.
PSNR, SSIM and grayscale intensity profiles provide additional ways to evaluate the results when suitable reference images are available.
Therefore, spatial filtering helps improve old photograph quality, but the kernel size must be selected carefully to preserve important facial features, hair patterns and historical details.

TASK 4: IMAGE SHARPENING AND EDGE ENHANCEMENT

1. Aim

To implement and compare different image sharpening and edge enhancement techniques for restoring faded and slightly blurred old photographs. The main objective is to improve image clarity, highlight important edges, and preserve facial features without increasing noise or creating artificial details.

2. Techniques Used

The following three sharpening techniques are considered:

1. **Laplacian Sharpening:** Uses second-order derivatives to identify rapid intensity changes and enhance edges.
2. **Unsharp Masking and High-Boost Filtering:** Enhances details by adding a scaled high-frequency component to the original image.
3. **Gradient-Based Sharpening:** Uses image gradients such as Sobel to identify and enhance edges.

3. Python Implementation

The following program implements all three techniques using OpenCV and NumPy.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import time

img = cv2.imread("old_photo.jpg", 0)

if img is None:
    raise ValueError("Image not found")
```

```
start = time.time()

# 1. Laplacian Sharpening
lap = cv2.Laplacian(img, cv2.CV_64F)
lap = np.uint8(np.clip(np.abs(lap), 0, 255))
lap_sharp = cv2.addWeighted(img, 1.0, lap, 0.7, 0)

# 2. Unsharp Masking
blur = cv2.GaussianBlur(img, (5, 5), 1.5)
unsharp = cv2.addWeighted(img, 1.5, blur, -0.5, 0)

# 3. High-Boost Filtering
high_boost = cv2.addWeighted(img, 2.0, blur, -1.0, 0)

# 4. Gradient-Based Sharpening
gx = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=3)
gy = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=3)

gradient = cv2.magnitude(gx, gy)
gradient = cv2.normalize(
    gradient, None, 0, 255, cv2.NORM_MINMAX
).astype(np.uint8)

gradient_sharp = cv2.addWeighted(img, 1.0, gradient, 0.5, 0)

end = time.time()

images = [
    img, lap_sharp, unsharp,
    high_boost, gradient_sharp
]

titles = [
    "Original Image",
    "Laplacian Sharpening",
```

```
    "Unsharp Masking",
    "High-Boost Filtering",
    "Gradient-Based Sharpening"
]

plt.figure(figsize=(12, 8))

for i in range(len(images)):
    plt.subplot(2, 3, i + 1)
    plt.imshow(images[i], cmap="gray")
    plt.title(titles[i])
    plt.axis("off")

plt.tight_layout()
plt.show()

print("Processing Time:", end - start, "seconds")

# Variance of Laplacian
for name, image in zip(titles, images):
    score = cv2.Laplacian(
        image, cv2.CV_64F
    ).var()
    print(name, "Sharpness:", score)
```

4. Comparison of Sharpening Techniques

Technique	Advantages	Limitations
Laplacian Sharpening	Highlights fine edges and small details	Can increase noise and create harsh edges
Unsharp Masking	Improves overall clarity and natural appearance	Excessive sharpening may create halos
High-Boost Filtering	Strong enhancement of edges and fine details	Can amplify scratches and grain
Gradient-Based Sharpening	Detects edges in horizontal and vertical directions	May not enhance smooth regions effectively

Laplacian sharpening is useful for identifying fine details. Unsharp masking provides balanced enhancement, while high-boost filtering produces stronger sharpening. Gradient-based sharpening is useful for highlighting important boundaries.

5. Sharpness Measurement Using Variance of Laplacian

The variance of the Laplacian is used to estimate image sharpness.

Formula:

[
S = Var(\nabla^2 I)
]

Where:

- (I) represents the input image.
- (\nabla^2 I) represents the Laplacian response.
- (Var) represents variance.

A higher value generally indicates stronger edge details and greater sharpness. However, noise and scratches can also increase this value. Therefore, it should not be used as the only measure of image quality.

The Python program calculates the variance of the Laplacian for the original and sharpened images. The results can be used to compare the sharpness of different techniques.

6. Intensity Profile Analysis

An intensity profile represents the pixel intensity values along a selected row or column of an image. It helps identify changes in brightness and edge sharpness.

```
row = img.shape[0] // 2

plt.figure(figsize=(10, 5))

plt.plot(img[row, :], label="Original")
plt.plot(lap_sharp[row, :], label="Laplacian")
plt.plot(unsharp[row, :], label="Unsharp")
plt.plot(high_boost[row, :], label="High-Boost")

plt.title("Intensity Profile Comparison")
plt.xlabel("Pixel Position")
plt.ylabel("Intensity")
plt.legend()
plt.grid()
```

plt.show()

The intensity profile helps observe how sharpening changes pixel values around edges. Sharper transitions may indicate improved edge definition. Excessively sharp peaks or sudden variations may indicate noise amplification or artificial enhancement.

7. Effect of Smoothing Before Sharpening

Smoothing before sharpening helps reduce unwanted noise and prevents the sharpening process from enhancing every small intensity variation.

Gaussian smoothing is commonly used before applying sharpening techniques.

Benefits:

1. Reduces random noise in the photograph.
2. Prevents excessive enhancement of grain and dust.
3. Produces smoother and more natural edges.
4. Improves the quality of the final restored image.

However, excessive smoothing may remove important details such as eyes, hair, facial boundaries, and fine textures. Therefore, a small Gaussian kernel such as 3×3 or 5×5 can be tested before sharpening.

8. Effect of Large High-Boost Amplification

High-boost filtering increases the strength of high-frequency image components. When the amplification factor becomes too large, unwanted image details may also become stronger.

For old photographs, this can cause:

- Scratches and dust marks to become more visible.
- Film grain and scanner noise to increase.
- Bright or dark halos around object boundaries.
- Unnatural facial textures.
- Loss of the original historical appearance.

Therefore, the amplification factor must be selected carefully. Moderate sharpening is preferred for preserving natural facial details.

9. Comparison with a Pretrained Deblurring Model

Traditional sharpening techniques improve edge visibility by enhancing high-frequency information. However, they cannot always recover details lost due to motion blur or severe defocus.

A pretrained image deblurring model attempts to remove blur using learned image patterns.

Traditional Sharpening	**Pretrained Deblurring Model**
Uses mathematical filters	Uses trained neural networks
Simple and fast	Usually requires more computation

Enhances existing edges	Attempts to recover blurred structures
May increase noise	May produce more natural-looking results
Does not reliably recover missing details	May generate or alter uncertain details

A pretrained deblurring model can be considered for severely blurred photographs. Its output must be checked carefully because it may introduce details that were not present in the original photograph.

10. Result and Conclusion

The Laplacian, Unsharp Masking, High-Boost, and Gradient-Based sharpening techniques were implemented using Python and OpenCV. Their effects can be compared using restored images, intensity profiles, processing time, and variance of Laplacian.

Unsharp masking is a suitable starting technique for improving the clarity of old photographs because it provides controlled sharpening. Laplacian sharpening is useful for highlighting fine edges, while high-boost filtering should be applied carefully to avoid increasing scratches and noise.

Conclusion: Image sharpening improves the visibility of important details in old photographs. However, excessive sharpening can damage the natural appearance of faces and backgrounds. A combination of light smoothing and moderate sharpening provides a balanced restoration approach. The final technique should be selected based on image quality, processing time, and preservation of original details.

TASK 5: FREQUENCY-DOMAIN LOW-PASS FILTERING

1. Aim

To implement and compare different frequency-domain low-pass filtering techniques for removing noise and unwanted patterns from old photographs while preserving important image details.

2. Techniques Used

The following three low-pass filters are implemented:

1. **Ideal Low-Pass Filter (ILPF):** Allows frequencies within a specified cutoff distance and removes frequencies outside it.
2. **Butterworth Low-Pass Filter (BLPF):** Gradually reduces high-frequency components based on filter order.

3. **Gaussian Low-Pass Filter (GLPF):** Uses a Gaussian function to reduce high-frequency components smoothly.

3. Python Implementation

The following program implements all three techniques using Python, NumPy, OpenCV, and Matplotlib.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import time

img = cv2.imread("old_photo.jpg", 0)

if img is None:
    raise ValueError("Image not found")

rows, cols = img.shape
crow, ccol = rows // 2, cols // 2

# Apply padding before Fourier Transform
padded = cv2.copyMakeBorder(
    img, 0, rows % 2, 0, cols % 2,
    cv2.BORDER_REFLECT
)

start = time.time()

# Fourier Transform
dft = np.fft.fft2(padded)
shift = np.fft.fftshift(dft)

u = np.arange(padded.shape[0]) - padded.shape[0] // 2
v = np.arange(padded.shape[1]) - padded.shape[1] // 2
U, V = np.meshgrid(v, u)

D = np.sqrt(U**2 + V**2)
```

```
D0 = 50

# Ideal Low-Pass Filter
ideal = np.where(D <= D0, 1, 0)

# Butterworth Low-Pass Filter
n = 2
butterworth = 1 / (1 + (D / D0)**(2*n))

# Gaussian Low-Pass Filter
gaussian = np.exp(-(D**2) / (2 * D0**2))

# Apply filters
ideal_result = np.fft.ifft2(
    np.fft.ifftshift(shift * ideal)
).real

butter_result = np.fft.ifft2(
    np.fft.ifftshift(shift * butterworth)
).real

gaussian_result = np.fft.ifft2(
    np.fft.ifftshift(shift * gaussian)
).real

# Crop to original image size
ideal_result = ideal_result[:rows, :cols]
butter_result = butter_result[:rows, :cols]
gaussian_result = gaussian_result[:rows, :cols]

# Normalize outputs
ideal_result = np.uint8(np.clip(ideal_result, 0, 255))
butter_result = np.uint8(np.clip(butter_result, 0, 255))
gaussian_result = np.uint8(np.clip(gaussian_result, 0, 255))
```

```
end = time.time()

images = [
    img, ideal_result,
    butter_result, gaussian_result
]

titles = [
    "Original Image",
    "Ideal Low-Pass",
    "Butterworth Low-Pass",
    "Gaussian Low-Pass"
]

plt.figure(figsize=(12, 7))

for i in range(4):
    plt.subplot(2, 2, i + 1)
    plt.imshow(images[i], cmap="gray")
    plt.title(titles[i])
    plt.axis("off")

plt.tight_layout()
plt.show()

# Display Fourier Spectrum
spectrum = np.log(1 + np.abs(shift))

plt.figure(figsize=(6, 5))
plt.imshow(spectrum, cmap="gray")
plt.title("Fourier Magnitude Spectrum")
plt.axis("off")
plt.show()
```

print("Processing Time:", end - start, "seconds")

4. Frequency Spectrum Analysis

The Fourier Transform converts an image from the spatial domain into the frequency domain.

- **Low-frequency components:** Represent smooth regions, gradual brightness changes, and overall illumination.
- **High-frequency components:** Represent sharp edges, fine textures, noise, and small image details.
- **Periodic patterns:** May appear as repeated bright peaks in the frequency spectrum.

In old photographs, scanner stripes and repeated patterns may produce visible frequency peaks. Identifying these patterns helps select suitable filtering techniques.

However, individual scratches and dust marks can contain a wide range of frequencies, so low-pass filtering cannot remove them completely without affecting useful details.

5. Effect of Different Cutoff Frequencies

The cutoff frequency determines how much image information is retained.

Cutoff Frequency	Effect
Low cutoff	Removes more high-frequency noise but produces greater blurring
Medium cutoff	Provides a balance between smoothing and detail preservation
High cutoff	Preserves more details but removes less noise

Different cutoff values such as 20, 50, and 100 can be tested to observe the changes in the restored photographs.

A suitable cutoff value should reduce unwanted noise while retaining important facial features, edges, and textures.

6. Comparison of Low-Pass Filters

Filter	Advantages	Limitations
Ideal Low-Pass	Simple implementation and clear cutoff control	Can produce ringing near sharp edges
Butterworth Low-Pass	Smooth transition between retained and rejected frequencies	Filter order affects smoothing and ringing
Gaussian Low-Pass	Smooth filtering with reduced ringing	May blur fine details and edges

The Ideal filter has a sudden cutoff, which may create ringing artifacts. The Butterworth filter provides a gradual transition. The Gaussian filter produces smoother results because its frequency response changes continuously.

7. Ringing, Fine Detail Loss and Edge Blurring

Low-pass filtering reduces high-frequency components, but excessive filtering can affect important image information.

Ringing Effect: The Ideal Low-Pass Filter may produce oscillations near sharp edges because of its abrupt frequency cutoff.

Fine Detail Loss: Small details such as hair, eyes, fabric patterns, and handwritten marks may disappear when too many high frequencies are removed.

Edge Blurring: Edges become less sharp because high-frequency information is reduced.

Therefore, the cutoff frequency should be selected carefully to maintain a balance between noise reduction and image clarity.

8. Comparison Between Frequency-Domain and Spatial-Domain Gaussian Filtering

Spatial-Domain Gaussian	Frequency-Domain Gaussian
Works directly on image pixels	Works on Fourier-transformed image data
Uses a Gaussian kernel	Uses a Gaussian frequency mask
Easy to implement using OpenCV	Requires Fourier Transform and inverse transform
Suitable for common image smoothing	Useful for analysing frequency patterns
Processing is generally simple	Requires additional transform operations

Both techniques can provide similar smoothing effects when their parameters and boundary handling are appropriately matched.

The main difference is the domain in which filtering is performed.

9. Effect of Not Using Padding Before DFT

Padding is important when applying frequency-domain filtering.

Without suitable padding, the Fourier Transform treats the image as periodic. This can cause opposite image boundaries to interact and produce unwanted edge artifacts.

The effects include:

1. Artificial discontinuities at image boundaries.
2. Unwanted ringing near edges.
3. Boundary-related artifacts in the filtered output.
4. Incorrect interpretation of image patterns.

Padding with reflected or replicated border pixels can reduce boundary discontinuities. Proper cropping should be performed after inverse transformation to obtain the original image dimensions.

10. Edge and Detail Retention Evaluation

The output images can be evaluated using edge and detail measurements.

A. Edge Retention

Sobel edge detection can be applied to the original and filtered images. The number and strength of detected edges can be compared.

B. Detail Preservation

Fine structures such as facial features and handwritten marks should be visually inspected before and after filtering.

C. Processing Time

The processing time of each filtering method can be recorded to identify which method is suitable for the available hardware.

The following code measures edge strength using the Sobel operator:

```
def edge_strength(image):
    gx = cv2.Sobel(
        image, cv2.CV_64F, 1, 0, ksize=3
    )

    gy = cv2.Sobel(
        image, cv2.CV_64F, 0, 1, ksize=3
    )

    magnitude = cv2.magnitude(gx, gy)

    return np.mean(magnitude)

for name, image in zip(titles, images):
    print(name, "Edge Strength:", edge_strength(image))
```

Higher edge strength does not always mean better quality because noise can also increase edge responses. Therefore, visual inspection and comparison with a clean reference image are important.

11. Result and Conclusion

The Ideal, Butterworth, and Gaussian Low-Pass Filters were implemented using the Fourier Transform. Their performance can be compared through filtered images, frequency spectrum analysis, edge strength, and processing time.

The Ideal filter provides direct frequency cutoff control but may introduce ringing. The Butterworth filter provides a gradual transition, while the Gaussian filter offers smooth noise reduction with fewer ringing artifacts.

Conclusion: Frequency-domain low-pass filtering is useful for reducing high-frequency noise and analysing repeated scanner patterns in old photographs. However, excessive filtering can remove important details and blur facial features. The Gaussian Low-Pass Filter is a suitable starting choice when smooth noise reduction and natural image appearance are important. The final cutoff frequency must be selected through testing on the same photographs.

TASK 6: FREQUENCY-DOMAIN HIGH-PASS FILTERING

1. Aim

To implement and compare frequency-domain high-pass filtering techniques for improving fine details and reducing uneven illumination effects in old photographs. The objective is to enhance important edges while analysing the effects of noise, brightness, and image quality.

2. Techniques Used

The following three techniques are considered:

1. **Ideal and Butterworth High-Pass Filters:** Remove low-frequency components and retain high-frequency details.
2. **Gaussian High-Pass Filter with High-Frequency Emphasis:** Enhances fine details by increasing the contribution of high-frequency components.
3. **Homomorphic Filtering:** Separates illumination and reflectance components to reduce uneven lighting and improve image contrast.

3. Python Implementation

The following program implements Ideal HPF, Butterworth HPF, Gaussian HPF, and Homomorphic Filtering.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import time

img = cv2.imread("old_photo.jpg", 0)

if img is None:
    raise ValueError("Image not found")

img = img.astype(np.float32) / 255.0
```

```
rows, cols = img.shape

start = time.time()

# Fourier Transform
dft = np.fft.fft2(img)
shift = np.fft.fftshift(dft)

u = np.arange(rows) - rows // 2
v = np.arange(cols) - cols // 2
U, V = np.meshgrid(v, u)

D = np.sqrt(U**2 + V**2)
D0 = 40

# Ideal High-Pass Filter
ideal_hpf = np.where(D >= D0, 1, 0)

# Butterworth High-Pass Filter
n = 2
butter_hpf = 1 / (1 + (D0 / (D + 1e-6))**(2*n))

# Gaussian High-Pass Filter
gaussian_hpf = 1 - np.exp(-(D**2) / (2 * D0**2))

# Apply High-Pass Filters
ideal_result = np.fft.ifft2(
    np.fft.ifftshift(shift * ideal_hpf)
).real

butter_result = np.fft.ifft2(
    np.fft.ifftshift(shift * butter_hpf)
).real

gaussian_result = np.fft.ifft2(
```

```
    np.fft.ifftshift(shift * gaussian_hpf)
).real

# Gaussian High-Frequency Emphasis
emphasis_result = 0.7 * img + 1.5 * gaussian_result

# Homomorphic Filtering
log_img = np.log1p(img)

log_dft = np.fft.fft2(log_img)
log_shift = np.fft.fftshift(log_dft)

gamma_L = 0.5
gamma_H = 1.8
c = 1.0

homomorphic_filter = (
    (gamma_H - gamma_L) *
    (1 - np.exp(-c * (D**2 / D0**2)))
    + gamma_L
)

filtered = np.fft.ifft2(
    np.fft.ifftshift(log_shift * homomorphic_filter)
).real

homomorphic_result = np.expm1(filtered)
homomorphic_result = np.clip(homomorphic_result, 0, 1)

end = time.time()

results = [
    img, ideal_result, butter_result,
    emphasis_result, homomorphic_result
]
```

```
titles = [
    "Original Image",
    "Ideal HPF",
    "Butterworth HPF",
    "Gaussian High-Frequency Emphasis",
    "Homomorphic Filtering"
]

plt.figure(figsize=(12, 8))

for i in range(len(results)):
    plt.subplot(2, 3, i + 1)
    plt.imshow(results[i], cmap="gray")
    plt.title(titles[i])
    plt.axis("off")

plt.tight_layout()
plt.show()

print("Processing Time:", end - start, "seconds")
```

4. Comparison of High-Pass Filtering Techniques

Technique	Advantages	Limitations
Ideal HPF	Highlights sharp edges and fine details	Can produce ringing artifacts
Butterworth HPF	Provides gradual frequency transition	May increase noise
Gaussian HPF	Produces smooth frequency enhancement	May reduce some low-frequency image information
Homomorphic Filtering	Reduces uneven illumination and improves contrast	Parameter selection is important

Ideal HPF removes frequencies below a fixed cutoff. Butterworth HPF provides a smoother transition, while Gaussian HPF gradually suppresses low frequencies.

Homomorphic filtering is particularly useful for photographs affected by uneven illumination because it adjusts the contribution of illumination and reflectance components.

5. Effect on Detail Sharpness, Uneven Fading and Noise

High-pass filtering enhances rapid intensity changes in an image.

Detail Sharpness: Fine structures such as hair, eyes, wrinkles, and photograph boundaries may become more visible.

Uneven Fading: Ordinary high-pass filters mainly emphasize edges and fine details. They do not directly correct uneven illumination. Homomorphic filtering is more suitable for reducing gradual brightness variations.

Noise Amplification: High-frequency noise, grain, scratches, and dust may also become stronger after high-pass filtering.

Therefore, high-pass filtering should be applied carefully to avoid damaging the original appearance of historical photographs.

6. Local Contrast and Intensity Variation Analysis

Local contrast refers to the difference in intensity between neighbouring image regions. High-pass filtering increases the visibility of rapid intensity changes.

The following program compares the intensity profiles of the original and processed photographs.

```
row = img.shape[0] // 2

plt.figure(figsize=(10, 5))

plt.plot(img[row, :], label="Original")
plt.plot(butter_result[row, :], label="Butterworth HPF")
plt.plot(emphasis_result[row, :], label="High-Frequency Emphasis")
plt.plot(homomorphic_result[row, :], label="Homomorphic")

plt.title("Intensity Profile Comparison")
plt.xlabel("Pixel Position")
plt.ylabel("Intensity")
plt.legend()
plt.grid()
plt.show()
```

The graph shows how pixel intensity changes across the selected image row. Stronger variations may indicate enhanced edges, while excessive variations can indicate noise or artificial enhancement.

7. Effect of Ideal High-Pass Filtering on Faces and Strong Edges

Ideal High-Pass Filtering removes low-frequency components and retains frequencies above the selected cutoff.

Its effects include:

1. Enhances strong boundaries and sharp transitions.
2. Highlights fine structures around eyes, hair, and facial outlines.
3. Removes gradual brightness information.
4. May create ringing near strong edges.
5. Can make faces appear unnatural if applied excessively.

Since facial appearance depends on both low-frequency and high-frequency information, applying only an Ideal HPF is not sufficient for complete photograph restoration.

A balanced approach should retain the original image and combine it with a controlled amount of high-frequency information.

8. Effect of Interchanging Low- and High-Frequency Gains in Homomorphic Filtering

Homomorphic filtering uses two gain parameters:

- (\gamma_L): Low-frequency gain.
- (\gamma_H): High-frequency gain.

Normally, the low-frequency gain is smaller than the high-frequency gain.

[
\gamma_L < 1,\qquad \gamma_H > 1
]

This reduces gradual illumination variations and enhances reflectance details.

If the gains are interchanged:

[
\gamma_L > 1,\qquad \gamma_H < 1
]

the filter increases low-frequency information and suppresses high-frequency information.

Effects of interchanging the gains:

1. Large illumination variations may become stronger.
2. Fine details and edges may become weaker.
3. The image may appear smoother.
4. Noise and small structures may be suppressed along with useful details.
5. The overall contrast distribution may change significantly.

Therefore, the usual gain arrangement is more appropriate when the objective is to reduce uneven illumination and enhance fine details.

9. Comparison with Pretrained Illumination Correction Models

Traditional homomorphic filtering uses mathematical frequency-domain operations. Pretrained illumination correction models use learned patterns to improve brightness and visibility.

Traditional Homomorphic Filtering	Pretrained Illumination Correction
Uses logarithmic transformation and Fourier filtering	Uses trained neural networks
Does not require training data	Requires a pretrained model
Offers direct parameter control	Learns complex illumination patterns
Can be computationally efficient	May require additional processing resources
May not handle complex lighting conditions effectively	May handle complex lighting variations better
Does not automatically reconstruct missing details	May alter uncertain image details

A pretrained illumination correction model can be tested on photographs with severe fading and uneven lighting. Its results should be checked to ensure that original facial features and historical details are preserved.

10. Result and Conclusion

Ideal HPF, Butterworth HPF, Gaussian High-Frequency Emphasis, and Homomorphic Filtering were implemented using Python and NumPy.

The techniques can be compared using processed images, intensity profiles, local contrast, noise visibility, and processing time.

High-pass filters are useful for enhancing fine details and strong edges. However, they may also increase scratches, grain, and unwanted noise. Homomorphic filtering is more suitable for correcting uneven illumination because it separates gradual lighting variations from image reflectance information.

Conclusion: Frequency-domain high-pass filtering improves the visibility of important image details. For old photograph restoration, homomorphic filtering provides a useful approach to reducing uneven illumination, while controlled high-frequency emphasis can improve edge visibility. The parameters should be selected carefully to maintain natural facial details and avoid excessive noise enhancement.

TASK 7: POINT, LINE AND EDGE DETECTION

1. Aim

To implement and compare different point, line, and edge detection techniques for identifying damaged regions in old photographs. The main objective is to detect dust particles, scratches, creases, stains, and torn boundaries while reducing false detections caused by noise.

2. Techniques Used

The following techniques are used:

1. **Point and Line Detection:** Uses special convolution masks to identify isolated points and line structures.
2. **Gradient-Based Edge Detection:** Uses Roberts, Prewitt, and Sobel operators to detect changes in image intensity.
3. **Marr-Hildreth and Canny Edge Detection:** Uses Laplacian of Gaussian and multi-stage edge detection to identify important image boundaries.

3. Python Implementation

The following program implements point, line, Roberts, Prewitt, Sobel, and Canny edge detection using Python and OpenCV.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("old_photo.jpg", 0)

if img is None:
    raise ValueError("Image not found")

# Point Detection
point_mask = np.array([
    [-1, -1, -1],
```

```
    [-1,  8, -1],
    [-1, -1, -1]
])

point_result = cv2.filter2D(
    img, cv2.CV_32F, point_mask
)

point_result = np.uint8(
    np.clip(np.abs(point_result), 0, 255)
)

# Line Detection
horizontal = np.array([
    [-1, -1, -1],
    [ 2,  2,  2],
    [-1, -1, -1]
])

vertical = np.array([
    [-1,  2, -1],
    [-1,  2, -1],
    [-1,  2, -1]
])

line_h = cv2.filter2D(img, cv2.CV_32F, horizontal)
line_v = cv2.filter2D(img, cv2.CV_32F, vertical)

line_result = np.uint8(
    np.clip(np.maximum(np.abs(line_h),
                       np.abs(line_v)), 0, 255)
)

# Roberts Edge Detection
roberts_x = np.array([
```

```
    [1, 0],
    [0, -1]
])

roberts_y = np.array([
    [0, 1],
    [-1, 0]
])

rx = cv2.filter2D(img, cv2.CV_32F, roberts_x)
ry = cv2.filter2D(img, cv2.CV_32F, roberts_y)

roberts = cv2.magnitude(rx, ry)
roberts = cv2.normalize(
    roberts, None, 0, 255, cv2.NORM_MINMAX
).astype(np.uint8)

# Prewitt Edge Detection
prewitt_x = np.array([
    [-1, 0, 1],
    [-1, 0, 1],
    [-1, 0, 1]
])

prewitt_y = np.array([
    [-1, -1, -1],
    [ 0,  0,  0],
    [ 1,  1,  1]
])

px = cv2.filter2D(img, cv2.CV_32F, prewitt_x)
py = cv2.filter2D(img, cv2.CV_32F, prewitt_y)

prewitt = cv2.magnitude(px, py)
prewitt = cv2.normalize(
```

```
    prewitt, None, 0, 255, cv2.NORM_MINMAX
).astype(np.uint8)

# Sobel Edge Detection
sx = cv2.Sobel(img, cv2.CV_32F, 1, 0, ksize=3)
sy = cv2.Sobel(img, cv2.CV_32F, 0, 1, ksize=3)

sobel = cv2.magnitude(sx, sy)
sobel = cv2.normalize(
    sobel, None, 0, 255, cv2.NORM_MINMAX
).astype(np.uint8)

# Canny Edge Detection
canny = cv2.Canny(img, 50, 150)

images = [
    img, point_result, line_result,
    roberts, prewitt, sobel, canny
]

titles = [
    "Original Image", "Point Detection",
    "Line Detection", "Roberts",
    "Prewitt", "Sobel", "Canny"
]

plt.figure(figsize=(12, 9))

for i in range(len(images)):
    plt.subplot(3, 3, i + 1)
    plt.imshow(images[i], cmap="gray")
    plt.title(titles[i])
    plt.axis("off")

plt.tight_layout()
```

```
plt.show()
```

4. Marr-Hildreth Edge Detection

The Marr-Hildreth method uses a Laplacian of Gaussian (LoG) filter to detect edges.

It involves the following steps:

1. Apply Gaussian smoothing to reduce noise.
2. Apply the Laplacian operator to identify intensity changes.
3. Detect zero-crossings in the filtered image.
4. Mark the detected boundaries as edges.

Python Implementation:

```
blur = cv2.GaussianBlur(img, (5, 5), 1.0)

log_result = cv2.Laplacian(
   blur, cv2.CV_32F, ksize=3
)

marr_hildreth = np.uint8(
   np.abs(log_result) > 20
) * 255

plt.imshow(marr_hildreth, cmap="gray")
plt.title("Marr-Hildreth Edge Detection")
plt.axis("off")
plt.show()
```

This is a simplified LoG response thresholding implementation. A complete Marr-Hildreth detector identifies zero-crossings between neighbouring positive and negative responses.

5. Detection of Damage in Old Photographs

Different edge and point detection techniques can help identify damaged areas.

Damage Type	Suitable Detection Technique
Dust particles	Point Detection
Thin scratches	Line Detection
Horizontal and vertical creases	Directional Line Masks
Torn boundaries	Sobel and Canny
Stains	Intensity and region analysis
Photograph boundaries	Canny Edge Detection

These techniques help locate damaged regions. However, detection alone does not restore the photograph. The detected areas must be reviewed before applying repair or inpainting methods.

6. Effect of Noise on Edge Detection

Noise can create unwanted intensity changes that appear similar to real edges. This may result in false detections.

Problems caused by noise:

1. Small dust particles may be detected as edges.
2. Scratches may produce multiple unwanted responses.
3. Important boundaries may be missed.
4. Random noise may create broken edge structures.
5. Facial details may be incorrectly classified as damage.

Applying Gaussian or median smoothing before edge detection can reduce noise. However, excessive smoothing may remove thin scratches and important image details.

7. Precision and Recall Evaluation

Precision and Recall are used to evaluate the accuracy of damage detection when manually annotated ground-truth masks are available.

Precision:

[

Precision = \frac{TP}{TP+FP}

]

Recall:

[

Recall = \frac{TP}{TP+FN}

]

Where:

- TP = Correctly detected damaged pixels.
- FP = Normal pixels incorrectly detected as damage.
- FN = Damaged pixels missed by the detector.

Metric	Meaning
High Precision	Fewer false damage detections
High Recall	More damaged areas correctly identified
Low Precision	Many normal regions are marked as damage
Low Recall	Important damaged areas are missed

Both values should be considered because a detector that identifies every small variation may have high recall but poor precision.

8. Threshold Selection for Different Damage Types

Threshold selection determines which image variations are classified as important points or edges.

Damage Type	Threshold Selection
Dust particles	Use a suitable point-response threshold
Thin scratches	Use lower line-response thresholds
Major boundaries	Use stronger edge thresholds
Faint creases	Use moderate thresholds with noise reduction

A lower threshold detects more weak edges but may increase false detections. A higher threshold reduces unwanted responses but may miss faded scratches and thin boundaries.

Therefore, different threshold values should be tested using the same photographs.

9. Limitations of Directional Line Detection

Directional line masks are designed to detect lines in specific orientations.

For example, a horizontal mask detects horizontal structures, while a vertical mask detects vertical structures.

Limitations:

1. Diagonal scratches may not be detected properly.
2. Curved scratches cannot be represented by a single directional mask.
3. Multiple orientations require additional masks.
4. Noise may create false line responses.
5. Complex damage patterns may require combined detection methods.

Using horizontal, vertical, and diagonal masks together can improve the detection of scratches with different orientations.

10. Comparison with Pretrained HED Edge Detection

Holistically-Nested Edge Detection (HED) is a deep-learning-based edge detection method. It uses a trained neural network to identify image boundaries at different levels.

Traditional Edge Detection	HED
Uses mathematical operators	Uses a pretrained neural network
Requires manual threshold selection	Learns edge patterns from training data
Simple and computationally efficient	Requires more computational resources
May detect noise as edges	Can produce more meaningful edge maps
Suitable for basic damage detection	Useful for complex image boundaries

HED can be tested as an additional method for detecting complex boundaries in old photographs. Its results should be reviewed because important facial details and damaged areas may both appear as edges.

11. Result and Conclusion

Point detection, line detection, Roberts, Prewitt, Sobel, Marr-Hildreth, and Canny techniques were implemented or demonstrated for identifying important image structures.

These methods help locate dust particles, scratches, creases, stains, and torn boundaries in old photographs. Their performance can be evaluated using detection maps, precision, recall, and visual comparison.

Conclusion: Point, line, and edge detection techniques are useful for identifying damaged regions in old photographs. Traditional methods are simple and efficient, but noise and complex damage patterns can reduce their accuracy. Combining suitable smoothing, directional masks, and edge detection techniques provides a better starting point for damage identification. Final detected regions should be reviewed before restoration to protect original facial features and historical details.

TASK 8: IMAGE THRESHOLDING AND BINARIZATION

1. Aim

To implement and compare different image thresholding techniques for converting old photographs into binary images. The main objective is to separate faded text, damaged regions, and important image details from the background.

2. Techniques Used

The following three thresholding techniques are considered:

1. **Global Thresholding:** Uses a single threshold value for the entire image.
2. **Otsu's Thresholding:** Automatically calculates an optimal threshold based on the image histogram.

3. **Adaptive Thresholding:** Calculates different threshold values for different local image regions.

3. Python Implementation

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("old_photo.jpg", 0)

if img is None:
    raise ValueError("Image not found")

# Global Thresholding
_, global_result = cv2.threshold(
    img, 120, 255, cv2.THRESH_BINARY
)

# Otsu's Thresholding
otsu_value, otsu_result = cv2.threshold(
    img, 0, 255,
    cv2.THRESH_BINARY + cv2.THRESH_OTSU
)

# Adaptive Thresholding
adaptive_result = cv2.adaptiveThreshold(
    img, 255,
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY,
    31, 10
)

images = [
    img, global_result,
    otsu_result, adaptive_result
]
```

```
titles = [
    "Original Image",
    "Global Threshold",
    "Otsu Threshold",
    "Adaptive Threshold"
]

plt.figure(figsize=(10, 7))

for i in range(4):
    plt.subplot(2, 2, i + 1)
    plt.imshow(images[i], cmap="gray")
    plt.title(titles[i])
    plt.axis("off")

plt.tight_layout()
plt.show()

print("Otsu Threshold Value:", otsu_value)
```

4. Comparison of Thresholding Techniques

Technique	Advantages	Limitations
Global Thresholding	Simple and fast	Not suitable for uneven illumination
Otsu's Thresholding	Automatically selects a threshold	May fail when foreground and background intensities overlap
Adaptive Thresholding	Handles local brightness variations	May produce noise if parameters are unsuitable

Global thresholding works well when the image has consistent brightness. Otsu's method is useful when the histogram has distinguishable foreground and background groups. Adaptive thresholding is more suitable for old photographs affected by shadows and uneven fading.

5. Performance on Lightly and Heavily Faded Photographs

Lightly faded photographs usually contain visible differences between the foreground and background. Global and Otsu thresholding may produce acceptable results in these conditions.

Heavily faded photographs have weak intensity differences. In such cases, global thresholding may miss faint details. Adaptive thresholding can perform better because it calculates thresholds for smaller image regions.

However, adaptive thresholding may also classify noise and stains as foreground objects. Therefore, suitable block sizes and constant values must be selected.

6. Binary Masks and Histogram Analysis

A binary mask represents the result of thresholding. White pixels represent the selected foreground, while black pixels represent the background.

Histograms help analyse the distribution of pixel intensities before and after thresholding.

```
plt.figure(figsize=(10, 5))

plt.hist(img.ravel(), 256, [0, 256],
        label="Original")

plt.hist(global_result.ravel(), 256, [0, 256],
        label="Global")

plt.hist(otsu_result.ravel(), 256, [0, 256],
        label="Otsu")

plt.title("Histogram Comparison")
plt.xlabel("Pixel Intensity")
plt.ylabel("Frequency")
plt.legend()
plt.show()
```

The histogram of a binary image mainly contains two intensity groups: 0 and 255. This helps understand how pixels are separated into foreground and background.

7. Dice Similarity Coefficient

The Dice Similarity Coefficient measures the similarity between the predicted binary mask and the manually prepared ground-truth mask.

Formula:

[
Dice = \frac{2|A \cap B|} {|A|+|B|}
]

Where:

- A = Predicted binary mask.
- B = Ground-truth mask.
- A ∩ B = Common foreground pixels.

Python Implementation:

```
def dice_score(predicted, ground_truth):
    predicted = predicted > 0
    ground_truth = ground_truth > 0

    intersection = np.logical_and(
        predicted, ground_truth
    ).sum()

    total = predicted.sum() + ground_truth.sum()

    return (2 * intersection) / (total + 1e-8)

ground_truth = cv2.imread(
    "ground_truth.png", 0
)

if ground_truth is not None:
    print("Global Dice:",
          dice_score(global_result, ground_truth))

    print("Otsu Dice:",
          dice_score(otsu_result, ground_truth))

    print("Adaptive Dice:",
          dice_score(adaptive_result, ground_truth))
```

A Dice score closer to 1 indicates better similarity. The ground-truth mask must be manually annotated and aligned with the input photograph for a valid comparison.

8. Why Global Thresholding Fails When Intensities Overlap

Global thresholding uses one threshold value for all pixels. It may fail when foreground and background intensities are similar.

For example:

1. Faded text may have nearly the same intensity as the paper.
2. Shadows may make background regions darker.
3. Stains may have intensities similar to important details.
4. Different areas may have different illumination levels.

As a result, some important details may disappear, while unwanted background regions may be selected.

Adaptive thresholding can reduce this problem by using local intensity information.

9. Effect of Adaptive Thresholding Parameters

Adaptive thresholding mainly depends on block size and constant value.

Parameter	Effect
Small Block Size	Detects local changes but may increase noise
Large Block Size	Produces smoother regions but may miss small details
Small Constant Value	Selects more foreground pixels in many cases
Large Constant Value	Changes foreground selection and may remove faint details

Block sizes such as 11, 21, and 31 can be tested. The constant value can also be adjusted based on the photograph's brightness and fading level.

10. Comparison with Deep-Learning-Based Damage Segmentation

Traditional thresholding separates pixels using intensity values. Deep-learning-based segmentation uses trained models to identify complex patterns and damaged regions.

Traditional Thresholding	Deep-Learning Segmentation
Uses pixel intensity values	Learns patterns from training images
Simple and fast	Requires more computational resources
Works well for clear intensity differences	Can handle complex damage patterns
Sensitive to uneven illumination	May handle different image conditions better
Requires parameter tuning	Requires suitable training data or a pretrained model

Deep-learning-based segmentation can be useful for identifying complex stains and scratches. However, its output must be checked to ensure that original image details are not incorrectly marked as damage.

11. Result and Conclusion

Global, Otsu, and Adaptive Thresholding techniques were implemented using Python and OpenCV. Their performance can be compared through binary masks, histograms, and Dice similarity scores.

Global thresholding is suitable for images with uniform illumination. Otsu's method automatically selects a threshold, while adaptive thresholding is useful for photographs with uneven lighting and fading.

Conclusion: Image thresholding and binarization are important steps in separating image details from the background. Adaptive thresholding is a useful starting choice for old photographs with uneven illumination. However, parameter selection and ground-truth evaluation are necessary to achieve reliable segmentation results.

TASK 9: REGION-BASED SEGMENTATION AND INTEGRATED PIPELINE DESIGN

1. Aim

To implement and compare region-based segmentation techniques for identifying damaged areas in old photographs. The objective is to detect stains, ink marks, dust clusters, creases, and torn regions and combine suitable image-processing techniques into an integrated restoration pipeline.

2. Techniques Used

The following three segmentation techniques are considered:

1. **Region Growing:** Starts from selected seed pixels and groups neighbouring pixels with similar intensity.
2. **Region Splitting and Merging:** Divides an image into smaller regions and combines similar neighbouring regions.
3. **Edge Linking and Boundary Detection:** Connects nearby edge pixels to identify continuous object or damage boundaries.

3. Python Implementation – Region Growing

Region growing begins with a seed point and checks neighbouring pixels. Pixels that satisfy the intensity similarity condition are added to the selected region.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from collections import deque
```

```
img = cv2.imread("old_photo.jpg", 0)

if img is None:
    raise ValueError("Image not found")

def region_growing(image, seed, threshold=15):
    rows, cols = image.shape
    mask = np.zeros((rows, cols), np.uint8)

    x, y = seed
    seed_value = int(image[y, x])

    queue = deque([(x, y)])
    mask[y, x] = 255

    while queue:
        cx, cy = queue.popleft()

        for dx, dy in [
            (-1, 0), (1, 0),
            (0, -1), (0, 1)
        ]:
            nx, ny = cx + dx, cy + dy

            if 0 <= nx < cols and 0 <= ny < rows:
                if mask[ny, nx] == 0:
                    difference = abs(
                        int(image[ny, nx]) - seed_value
                    )

                    if difference <= threshold:
                        mask[ny, nx] = 255
                        queue.append((nx, ny))

    return mask
```

```
# Example seed point
seed = (img.shape[1] // 2, img.shape[0] // 2)

region_result = region_growing(img, seed, 15)

plt.figure(figsize=(10, 5))

plt.subplot(1, 2, 1)
plt.imshow(img, cmap="gray")
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(region_result, cmap="gray")
plt.title("Region Growing")
plt.axis("off")

plt.show()
```

The seed point should be selected inside the region that needs to be segmented. This basic implementation compares pixels with the initial seed intensity.

4. Region Splitting and Merging

Region splitting divides an image into smaller sections based on intensity variation. Regions with similar properties can then be merged.

Main Steps:

1. Start with the complete image as one region.
2. Calculate the intensity variation within the region.
3. Split regions that have high variation.
4. Compare neighbouring regions.
5. Merge regions with similar intensity values.
6. Generate the final segmentation mask.

A quadtree structure can be used to divide an image into four smaller regions repeatedly until the required homogeneity condition is satisfied.

Advantages:

- Identifies large stains and uneven regions.

- Handles images with different intensity areas.
- Provides a structured segmentation approach.

Limitations:

- Incorrect splitting may create too many small regions.
- Similar regions may be difficult to separate.
- Parameter selection affects the final segmentation.

5. Edge Linking and Boundary Detection

Edge linking connects nearby edge pixels to form continuous boundaries. It is useful for identifying scratches, creases, and torn photograph regions.

```
# Edge detection
edges = cv2.Canny(img, 50, 150)

# Morphological closing to connect nearby edges
kernel = np.ones((3, 3), np.uint8)

linked_edges = cv2.morphologyEx(
    edges,
    cv2.MORPH_CLOSE,
    kernel
)

plt.figure(figsize=(10, 5))

plt.subplot(1, 2, 1)
plt.imshow(edges, cmap="gray")
plt.title("Canny Edges")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(linked_edges, cmap="gray")
plt.title("Linked Edges")
plt.axis("off")

plt.show()
```

Morphological closing connects small gaps between nearby edge pixels. However, a large kernel may incorrectly connect separate boundaries.

6. Detection of Different Damaged Regions

Damage Type	Suitable Segmentation Technique
Stains	Region Growing
Ink Marks	Region Growing and Thresholding
Dust Clusters	Thresholding and Region-Based Segmentation
Creases	Edge Linking
Torn Boundaries	Canny Edge Detection and Edge Linking
Large Damaged Areas	Region Splitting and Merging

Combining multiple techniques can improve damage identification because different types of damage have different shapes and intensity patterns.

7. Evaluation Using IoU and Boundary Localization Error

Intersection over Union (IoU) measures the overlap between the predicted damage region and the ground-truth mask.

Formula:

[
IoU = \frac{|A \cap B|}{|A \cup B|}
]

Where:

- A = Predicted segmentation mask.
- B = Ground-truth mask.

A higher IoU indicates better segmentation accuracy.

Boundary Localization Error: Measures the distance between the detected boundary and the manually annotated reference boundary. A smaller error indicates more accurate boundary detection.

Both metrics require manually annotated ground-truth data.

8. Integrated Old Photograph Restoration Pipeline

An integrated pipeline combines the most suitable techniques from the previous tasks.

Proposed Pipeline:

1. Input photograph.
2. Image resizing and preprocessing.
3. Noise reduction using Gaussian or Median filtering.
4. Contrast enhancement using CLAHE.

5. Edge and damage detection.
6. Region-based segmentation.
7. Damage mask generation.
8. Manual verification of detected areas.
9. Restoration using suitable inpainting techniques.
10. Final quality evaluation and image output.

Block Diagram:

Input Old Photograph

|

v

Image Preprocessing

|

v

Noise Reduction

|

v

Contrast Enhancement

|

v

Edge and Damage Detection

|

v

Region-Based Segmentation

|

v

Damage Mask Creation

|

v

Manual Mask Verification

|

v

Image Restoration / Inpainting

|

v

Quality Evaluation

|

v

Restored Photograph

The original photograph should always be preserved separately. Reconstructed areas must be identified so that generated details are not mistaken for original historical information.

9. Processing Time Evaluation

Processing time should be measured for every stage of the integrated pipeline.

Processing Stage	Measurement
Image Preprocessing	Time in seconds
Noise Reduction	Time in seconds
Contrast Enhancement	Time in seconds
Edge Detection	Time in seconds
Region Segmentation	Time in seconds
Image Restoration	Time in seconds
Final Evaluation	Time in seconds

The assignment specifies a 5-second processing target for this task, while the operating scenario gives a maximum of 3 seconds per page. Both requirements should be reported separately. The stricter 3-second overall target should be followed unless the assignment explicitly changes the operating requirement.

Python Timing Example:

```
import time

start = time.time()

# Insert the complete restoration pipeline here

end = time.time()

print("Total Processing Time:",
    end - start, "seconds")
```

Actual timings should be measured on the available computer using the same input images and hardware.

10. Effect of Incorrect Seed Point Selection

Region growing depends strongly on the selected seed point.

If the seed point is placed in the wrong area:

1. An unwanted region may be selected.
2. Important image details may be included in the mask.
3. The damaged region may not be detected correctly.
4. Segmentation accuracy may decrease.
5. Restoration may affect normal image areas.

To reduce this problem, seed points should be selected carefully. Multiple seed points or automatic seed selection can also be tested.

11. Problems with Overlapping and Interrupted Boundaries

Overlapping and interrupted boundaries make segmentation difficult.

Common Problems:

- Scratches may cross important facial features.
- Torn boundaries may contain missing sections.
- Multiple stains may overlap.
- Noise may create gaps in detected edges.
- Nearby boundaries may be incorrectly connected.

Combining edge detection with region-based segmentation can improve the results. Morphological operations may connect small gaps, but their parameters must be selected carefully to avoid merging unrelated regions.

12. Comparison with Segment Anything Model 2 (SAM 2)

SAM 2 is a promptable segmentation model that can identify objects and regions using suitable prompts.

Traditional Segmentation	**SAM 2**
Uses intensity and edge information	Uses a trained deep-learning model
Requires manual parameter selection	Can use prompts to guide segmentation
Computationally simple	Requires more computational resources
Suitable for basic image regions	Can handle complex object boundaries
Sensitive to noise and intensity changes	May identify meaningful object regions
Does not require a pretrained model	Requires model weights and suitable hardware

SAM 2 can be tested for identifying large damaged regions and complex boundaries. However, it is not specifically designed for historical photograph damage detection, so its masks must be checked manually.

13. Result and Conclusion

Region Growing, Region Splitting and Merging, and Edge Linking techniques were studied and implemented or demonstrated for segmenting damaged areas in old photographs.
The techniques can be evaluated using segmentation masks, IoU, boundary localization error, and processing time.
Conclusion: Region-based segmentation helps identify stains, dust clusters, ink marks, creases, and torn boundaries. Combining region segmentation with edge detection and image enhancement provides a useful restoration pipeline. Manual verification is important to prevent damage to original facial details and historical information.

TASK 10: RESTORATION SYSTEM ADAPTATION FOR COLOUR AND HIGH-RESOLUTION PHOTOGRAPHS

1. Aim

To adapt the existing old photograph restoration system for high-resolution colour photographs captured using mobile phones and scanners. The objective is to correct colour fading, remove noise and damage, preserve facial details, and maintain the original appearance of historical photographs.

2. Challenges in Colour and High-Resolution Photographs

The restoration system must handle photographs with different types of damage.

Major Challenges:

1. Yellow and sepia colour changes.
2. Faded facial features and background details.
3. Scratches, stains, and dust marks.
4. JPEG compression and blocking artifacts.
5. High-resolution images requiring more memory and processing time.
6. Handwritten dates on the reverse side that must not be removed.

The system should process photographs of approximately 4000 × 3000 pixels while maintaining good image quality and completing processing within 5 seconds on the specified computer.

3. Three System Adaptation Approaches

The following three approaches are considered.

A. RETAIN – Existing Restoration Pipeline

The existing grayscale restoration pipeline can be retained with minimal changes.

- Convert colour images into grayscale.
- Apply noise reduction.
- Perform contrast enhancement.
- Detect damaged regions.

- Apply suitable restoration techniques.

Advantages:

- Simple implementation.
- Reuses existing methods.
- Requires fewer modifications.

Limitations:

- Original colour information is lost.
- Yellow and sepia colour casts cannot be corrected properly.
- Colour details of faces, clothes, and backgrounds are not preserved.

B. MODIFY – Improve the Existing Pipeline

The existing pipeline can be modified to process luminance and colour information separately.

- Convert the image into Lab or HSV colour space.
- Improve brightness and contrast using the luminance channel.
- Adjust colour channels to reduce yellow and sepia casts.
- Apply noise reduction carefully.
- Detect scratches and stains.
- Preserve important details using masks.

Advantages:

- Preserves colour information.
- Improves brightness and contrast.
- Requires fewer changes than a complete redesign.
- Supports existing restoration techniques.

Limitations:

- Colour correction requires suitable parameter selection.
- Incorrect adjustments may produce unnatural colours.
- High-resolution images may require additional processing time.

C. REDESIGN – Complete Colour Restoration System

A complete redesign can introduce advanced colour restoration, frequency-domain processing, damage segmentation, and inpainting models.

- Perform automatic colour correction.
- Separate luminance and chrominance information.
- Detect damage using segmentation techniques.
- Reduce noise and JPEG artifacts.
- Restore damaged regions using inpainting.
- Evaluate the final output using image-quality metrics.

Advantages:

- Supports complex colour restoration.
- Can handle multiple types of damage.
- Provides opportunities for advanced restoration models.

Limitations:

- Requires more development time.
- May need additional training data or pretrained models.
- Can increase memory usage and processing time.
- Generative restoration may introduce details that were not present originally.

4. Python Implementation – Colour Restoration Approaches

The following program demonstrates the three approaches using OpenCV.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import time

img = cv2.imread("old_color_photo.jpg")

if img is None:
    raise ValueError("Image not found")

start = time.time()

# A. RETAIN: Convert to grayscale
gray = cv2.cvtColor(
    img, cv2.COLOR_BGR2GRAY
)

retain = cv2.equalizeHist(gray)

# B. MODIFY: Lab-based enhancement
lab = cv2.cvtColor(
    img, cv2.COLOR_BGR2LAB
)
```

```
l, a, b = cv2.split(lab)

clahe = cv2.createCLAHE(
    clipLimit=2.0,
    tileGridSize=(8, 8)
)

enhanced_l = clahe.apply(l)

modified_lab = cv2.merge(
    [enhanced_l, a, b]
)

modify = cv2.cvtColor(
    modified_lab, cv2.COLOR_LAB2BGR
)

# C. REDESIGN: Colour correction
b_channel, g_channel, r_channel = cv2.split(
    img.astype(np.float32)
)

mean_b = np.mean(b_channel)
mean_g = np.mean(g_channel)
mean_r = np.mean(r_channel)

target = (mean_b + mean_g + mean_r) / 3

b_channel *= target / (mean_b + 1e-6)
g_channel *= target / (mean_g + 1e-6)
r_channel *= target / (mean_r + 1e-6)

corrected = cv2.merge([
    b_channel, g_channel, r_channel
])
```

```
corrected = np.clip(
    corrected, 0, 255
).astype(np.uint8)

redesign = cv2.fastNlMeansDenoisingColored(
    corrected, None, 5, 5, 7, 21
)

end = time.time()

# Display Results
images = [
    cv2.cvtColor(img, cv2.COLOR_BGR2RGB),
    retain,
    cv2.cvtColor(modify, cv2.COLOR_BGR2RGB),
    cv2.cvtColor(redesign, cv2.COLOR_BGR2RGB)
]

titles = [
    "Original Image",
    "Retain",
    "Modify",
    "Redesign"
]

plt.figure(figsize=(12, 8))

for i in range(4):
    plt.subplot(2, 2, i + 1)
    plt.imshow(images[i], cmap="gray" if i == 1 else None)
    plt.title(titles[i])
    plt.axis("off")

plt.tight_layout()
```

```
plt.show()


print("Processing Time:",
      end - start, "seconds")
```

This is a basic comparison implementation. The colour correction step uses channel means and may not produce accurate colour for every photograph. The redesign approach can be extended with damage segmentation and inpainting. Actual performance must be tested on the selected dataset.

5. Comparison of the Three Approaches

Evaluation Factor	RETAIN	MODIFY	REDESIGN
Colour Correction	Not supported properly	Supported	Advanced correction possible
Detail Preservation	Limited	Good with careful tuning	Depends on model and processing
Noise Reduction	Basic	Improved	Advanced methods possible
Damage Detection	Existing methods	Improved methods	Advanced segmentation possible
Facial Preservation	Limited colour information	Preserves colour and details	Requires careful verification
Background Preservation	Basic	Good	Depends on restoration method
Development Effort	Low	Medium	High

The RETAIN approach is suitable for grayscale photographs. MODIFY is more suitable for colour photographs because it preserves colour information while reusing existing techniques. REDESIGN offers advanced possibilities but requires more resources and testing.

6. Evaluation Metrics

The following metrics can be used to evaluate the restoration system.

A. Dice Similarity Coefficient

Used to compare predicted damage masks with manually annotated ground-truth masks.

[
Dice = \frac{2|A \cap B|} {|A|+|B|}
]

B. Colour Fidelity

Colour accuracy can be measured using the CIE Delta E value when a suitable reference image is available. A smaller colour difference indicates better colour similarity.

C. Edge Preservation

Edge detection can be applied to the original and restored photographs to check whether important facial and background boundaries are retained.

D. PSNR

Peak Signal-to-Noise Ratio measures the difference between a restored image and a clean reference image.

[
PSNR = 10\log_{10}
\left(\frac{MAX^2}{MSE}\right)
]

E. SSIM

Structural Similarity Index measures similarity in image structure, contrast, and brightness.

F. Processing Time

The total time required to restore each photograph should be measured to verify the 5-second processing requirement.

PSNR and SSIM require a suitable clean reference image. Dice requires an annotated damage mask. These metrics should not be reported as measured results unless the required reference data is available.

7. Dataset and Testing Procedure

The system should be tested using at least 15 high-resolution colour photographs.

Testing Procedure:

1. Collect at least 15 old colour photographs.
2. Record image dimensions and file formats.
3. Identify fading, scratches, stains, and compression artifacts.
4. Apply RETAIN, MODIFY, and REDESIGN approaches.
5. Compare the restored images.
6. Calculate suitable quality metrics.
7. Record processing time for every photograph.
8. Identify which approach provides the best balance between quality and speed.

The same input photographs and hardware should be used for all three approaches to ensure a fair comparison.

8. Improved Restoration Pipeline

Based on the comparison, the MODIFY approach is selected as the practical starting design for colour photograph restoration.

Proposed Improved Pipeline:

High-Resolution Colour Image

|

v

Image Loading and Checking

|

v

Colour Space Conversion

|

v

Luminance Enhancement

|

v

Colour Correction

|

v

Noise Reduction

|

v

Damage Detection

|

v

Damage Segmentation

|

v

Manual Mask Verification

|

v

Inpainting and Restoration

|

v

Colour and Detail Evaluation

|

v

Final Restored Image

The pipeline should preserve the original image and use separate masks for damaged regions. Reverse-side handwritten dates should be protected by excluding them from automatic damage removal.

9. Possible Failure Conditions

The restoration system may fail under certain conditions.

1. Severe fading may remove important image information.
2. Large torn regions may contain missing details.
3. Incorrect damage masks may remove facial features.
4. Strong yellow or sepia colour casts may cause inaccurate colour correction.
5. JPEG compression may permanently affect fine details.
6. Excessive inpainting may generate artificial structures.
7. High-resolution processing may exceed the required time or memory limits.

To reduce these problems, the system should use manual verification, moderate restoration parameters, and preservation of the original photograph.

10. Restoration Models and Risk of Inventing Details

Deep-learning-based restoration models can improve damaged photographs by learning patterns from training images.

Examples include:

- Image denoising models for removing noise.
- Image deblurring models for reducing blur.
- Inpainting models for filling damaged regions.
- Colour restoration models for improving faded colours.

However, generative inpainting may create plausible details in missing regions. These details may not match the original photograph.

Precautions:

1. Preserve the original photograph separately.
2. Use damage masks to control the restoration area.
3. Avoid modifying undamaged facial features.
4. Keep handwritten dates and historical text protected.
5. Mark reconstructed regions clearly.
6. Compare restored outputs with the original image.

Restoration should improve visibility without presenting invented details as historically accurate information.

11. Final Decision: RETAIN, MODIFY or REDESIGN

<underline>Selected Approach: MODIFY</underline>

The MODIFY approach is selected because it provides a balance between colour preservation, restoration quality, processing time, and development effort.

Approach	Final Decision	Reason
RETAIN	Not Selected	Cannot properly preserve colour information
MODIFY	Selected	Preserves colour and reuses existing techniques
REDESIGN	Future Improvement	Requires more resources and testing

The selected approach can be upgraded later with advanced segmentation and restoration models if the available hardware and processing time permit.

12. Result and Conclusion

The RETAIN, MODIFY, and REDESIGN approaches were studied for adapting the existing restoration system to high-resolution colour photographs. Their performance can be evaluated using colour fidelity, Dice score, PSNR, SSIM, edge preservation, and processing time.

The MODIFY approach is selected as the most practical starting solution because it preserves colour information and improves the existing grayscale pipeline without requiring a complete redesign.

Conclusion: A successful colour photograph restoration system must correct fading, reduce noise, identify damaged regions, and preserve original facial and historical details. The MODIFY approach provides a balanced solution for the given requirements. Advanced restoration models can be added in the future, but reconstructed details must always be identified and verified to avoid misleading historical results.